# COMPUTER NETWORKS

## Complete Placement Preparation Guide

12 Chapters · 50 Interview Questions · 25 Practice Problems

From Physical Layer to CDNs — Every Concept You Need for SDE Interviews

Topics: OSI & TCP/IP Models · Subnetting & VLSM · TCP/UDP Deep Dive · DNS · HTTP/HTTPS/TLS · Routing Algorithms · Network Security · Wireless Networks · Load Balancing · CDN Architecture

---

*2024 Edition — For Campus Placements & Tech Interviews*

# TABLE OF CONTENTS

# CHAPTER 1: INTRODUCTION & NETWORK FUNDAMENTALS

## 1.1 What is a Computer Network?

A **computer network** is a collection of autonomous computing devices (nodes) interconnected by communication links and switches, configured to enable data exchange and resource sharing. The word 'autonomous' is critical — each device operates independently; a network is not the same as a parallel or distributed computer where devices share a single OS.

The four fundamental goals of networking are: **Resource Sharing** — hardware (printers, storage), software (applications), and data are available to all users regardless of physical location; **Communication** — enables email, VoIP, video conferencing, and real-time collaboration; **Reliability** — data can travel via alternate paths if one fails (fault tolerance); **Cost Reduction** — shared resources eliminate the need for every node to have expensive peripherals.

## 1.2 Network Criteria: Performance, Reliability, Security

**Bandwidth**: Maximum theoretical data rate a medium can carry (bps).

**Throughput**: Actual data transferred per second in practice (always ≤ bandwidth).

**Latency/Delay**: Time from sender to receiver. Components: propagation delay, transmission delay, queuing delay, processing delay.

**Jitter**: Variation in packet arrival times — critical for real-time audio/video.

**Packet Loss**: % of packets dropped; caused by congestion, link errors, or queue overflow.

**Transmission Delay** = Packet Size (bits) ÷ Bandwidth (bps). **Propagation Delay** = Distance ÷ Propagation Speed (~2×10■ m/s in copper). These two are frequently confused in interviews. Transmission delay depends on bandwidth; propagation delay depends only on distance.

## 1.3 Types of Networks

| Type | Full Name | Scale | Example / Technology |
|---|---|---|---|
| PAN | Personal Area Network | ~10 m | Bluetooth, USB |
| LAN | Local Area Network | 100m – 1km | Ethernet, Wi-Fi (home/office) |
| CAN | Campus Area Network | 1 – 5 km | University / corporate campus |
| MAN | Metropolitan Area Network | 5 – 50 km | Cable TV, city fiber ring |
| WAN | Wide Area Network | Nationwide/Global | Internet, MPLS, leased lines |
| WLAN | Wireless LAN | Same as LAN | IEEE 802.11 (Wi-Fi) |
| SAN | Storage Area Network | Data center | Fibre Channel, iSCSI |
| VPN | Virtual Private Network | Any distance | IPsec / SSL over Internet |

## 1.4 Network Topologies

A **network topology** describes how devices (nodes) are physically or logically arranged and connected.

| Topology | Description | Pros | Cons |
|---|---|---|---|
| Bus | Single shared cable; all devices tap in | Simple, low cost | Single point of failure; collision domain |
| Star | Central hub/switch; all devices connect to it | Easy to add/remove nodes; isolated failures | Hub/switch = single failure point |
| Ring | Each node connects to exactly two others | Equal access; no collisions with token | One break can halt the entire ring |

| Topology | Description | Pros | Cons |
|---|---|---|---|
| Mesh | Every node connected to every other (full) or some (partial) | High fault tolerance; multiple paths | Expensive; complex; n(n-1)/2 links for full mesh |
| Tree | Hierarchical; root → branches → leaves | Scalable; structured | Root failure breaks subtree |
| Hybrid | Combination of two or more topologies | Flexible; scalable | Complex design and management |

## 1.5 Transmission Modes & Network Devices

**Simplex**: One-directional only. E.g., keyboard → CPU, TV broadcast.

**Half-Duplex**: Both directions, but not simultaneously. E.g., walkie-talkie, CSMA/CD Ethernet.

**Full-Duplex**: Both directions simultaneously. E.g., telephone, modern Ethernet switches.

| Device | OSI Layer | Function / Key Point |
|---|---|---|
| Repeater | L1 Physical | Amplifies/regenerates signal; extends cable distance |
| Hub | L1 Physical | Multi-port repeater; broadcasts to all ports; creates one collision domain |
| Bridge | L2 Data Link | Connects two LAN segments; filters by MAC address; reduces collisions |
| Switch | L2 Data Link | Multi-port bridge; creates separate collision domain per port; uses MAC table |
| Router | L3 Network | Connects different networks; routes by IP address; separates broadcast domains |
| Gateway | L4–L7 | Protocol translator; connects networks using different protocols (e.g., IP to SNA) |
| Access Point | L2 Data Link | Wireless entry point; bridges wireless to wired LAN (IEEE 802.11) |
| NIC | L1+L2 | Network Interface Card; has MAC address; converts digital data to network signals |
| Modem | L1 Physical | MOdulator-DEModulator; converts digital ↔ analog for transmission over phone line |
| Firewall | L3–L7 | Enforces security policy; packet filtering, stateful inspection, application proxy |

## 1.6 Switching Techniques

**Switching** is the mechanism used to move data from source to destination across a network. Three main paradigms exist:

| Feature | Circuit Switching | Packet Switching (Datagram) | Message Switching |
|---|---|---|---|
| Path Setup | Dedicated path established before data sent | No pre-established path; each packet routed independently | Path established per message; store-and-forward |
| Resource | Resources reserved for entire call | Resources shared dynamically | Entire message stored at each node |
| Delay | Fixed delay; no queuing after setup | Variable delay; queuing at each router | High delay; message stored at each hop |
| Bandwidth | Wasted if idle | Efficient; statistical multiplexing | Inefficient; large storage needed |
| Example | PSTN telephone | Internet (IP) | Email (historically) |
| Packet Loss | None (circuit reserved) | Possible; routers may drop packets | None (message stored until delivered) |

Within Packet Switching, **Datagram** mode routes each packet independently (Internet), while **Virtual Circuit** mode establishes a path once and all packets follow it (ATM, Frame Relay) — giving it properties between circuit and datagram switching.

**■ INTERVIEW TIP — Circuit vs Packet Switching**

Interviewers love: 'Why does the Internet use packet switching?' Answer: efficient bandwidth use via statistical multiplexing, resilience to link failures (alternate paths), and suitability for bursty data traffic. Circuit switching wastes bandwidth when the circuit is idle.

## ■ QUICK REVISION BOX

◆ Network Goals: Resource sharing, Communication, Reliability, Cost reduction

◆ Performance = Bandwidth vs Throughput vs Latency vs Jitter vs Packet Loss

◆ Hub (L1, shared collision domain) vs Switch (L2, per-port collision domain) vs Router (L3, separates broadcast domains)

◆ LAN < CAN < MAN < WAN in scale; Wi-Fi is WLAN using IEEE 802.11

◆ Circuit Switching: dedicated path, guaranteed BW; Packet Switching: shared, efficient, variable delay

◆ Full-Duplex = simultaneous bi-directional (switches); Half-Duplex = one at a time (walkie-talkie, old Ethernet hubs)

# CHAPTER 2: THE OSI MODEL & TCP/IP MODEL

## 2.1 OSI 7-Layer Model

The **Open Systems Interconnection (OSI) model** is a conceptual framework standardised by ISO (1984) to describe how different network protocols interact. It divides network communication into 7 distinct layers, each responsible for specific functions. The model ensures that components from different vendors can interoperate.

```
OSI LAYER STACK +-------+-----------------+----------+------------------+ | Layer |
Name | PDU | Example Protocol |
+-------+-----------------+----------+-------------------+ | 7 | Application | Data |
HTTP, FTP, DNS | | 6 | Presentation | Data | TLS/SSL, JPEG | | 5 | Session | Data |
NetBIOS, RPC | | 4 | Transport | Segment | TCP, UDP | | 3 | Network | Packet | IP, ICMP,
OSPF | | 2 | Data Link | Frame | Ethernet, PPP | | 1 | Physical | Bit | RS-232, 802.11 |
+-------+-----------------+----------+------------------+
```

### *Layer 7 — Application Layer*

Provides network services directly to end-user applications. It is not an application itself but the interface through which applications access network services. Protocols: **HTTP** (web), **FTP** (file transfer), **SMTP** (email), **DNS** (name resolution), **DHCP** (IP assignment), **SNMP** (network management), Telnet, SSH.

### *Layer 6 — Presentation Layer*

Responsible for **data translation** (e.g., ASCII ↔ EBCDIC), **encryption/decryption** (TLS/SSL operates here), and **compression**. It ensures data sent by one application can be read by another on a different system. Formats: JPEG, MPEG, ASCII, EBCDIC, GIF.

### *Layer 5 — Session Layer*

Manages **sessions** (logical connections) between applications. Responsible for **dialog control** (half-duplex vs full-duplex), **synchronisation** (checkpointing for long transfers so recovery is possible), and session establishment/maintenance/termination. Protocols: NetBIOS, RPC, PPTP.

### *Layer 4 — Transport Layer*

Provides **process-to-process** (end-to-end) communication using **port numbers**. Responsible for **segmentation and reassembly**, **flow control** (prevents sender from overwhelming receiver), **congestion control** (prevents network overload), and **error control** (retransmission). Protocols: **TCP** (reliable, connection-oriented), **UDP** (unreliable, connectionless).

### *Layer 3 — Network Layer*

Provides **logical addressing** (IP addresses) and **routing** — determining the best path from source to destination across multiple networks. Also handles **fragmentation/reassembly** when packets exceed MTU of a link. Protocols: **IP (IPv4/IPv6)**, ICMP, IGMP, ARP (sometimes placed here), OSPF, RIP, BGP.

### *Layer 2 — Data Link Layer*

Provides **node-to-node** (hop-to-hop) data transfer over a single link. Responsible for **framing**, **MAC addressing**, **error detection** (CRC), **flow control**, and **media access control**. Split into two sublayers: **LLC (Logical Link Control)** — flow/error control; **MAC (Medium Access Control)** — addresses and media access. Protocols: Ethernet (IEEE 802.3), Wi-Fi (IEEE 802.11), PPP, HDLC.

### *Layer 1 — Physical Layer*

Deals with the raw **bit stream** — electrical, optical, or radio signals. Responsible for **bit representation** (encoding), **data rate**, **synchronisation**, **transmission mode**, and physical connection (connectors, cables). Key concepts: bandwidth, baud rate (symbols/sec ≠ bits/sec), modulation. Standards: RS-232, RJ-45, IEEE 802.3, DSL.

## 2.2 TCP/IP Model

The **TCP/IP model** (also called the Internet model) was developed by DARPA before the OSI model and is what the Internet actually uses. It has either 4 or 5 layers depending on the textbook. The 5-layer version splits the bottom into Data Link + Physical; the 4-layer version merges them into 'Network Access.'

```
TCP/IP 5-LAYER MODEL TCP/IP 4-LAYER MODEL +------------------+ +------------------+ | 5.
Application | | 4. Application | <- HTTP, DNS, SMTP +------------------+
+------------------+ | 4. Transport | | 3. Transport | <- TCP, UDP +------------------+
+------------------+ | 3. Internet | | 2. Internet | <- IP, ICMP +------------------+
+------------------+ | 2. Data Link | | 1. Net Access | <- Ethernet, Wi-Fi
+------------------+ +------------------+ | 1. Physical | +------------------+
```

## 2.3 OSI vs TCP/IP — Detailed Comparison

| Feature | OSI Model | TCP/IP Model |
|---|---|---|
| Layer Count | 7 layers | 4 or 5 layers |
| Developed by | ISO (1984) | DARPA (1970s) |
| Application Layer | App + Presentation + Session (3 layers) | Single Application layer |
| Transport Layer | Layer 4 — TCP/UDP | Layer 3 (4-layer) — TCP/UDP |
| Network Layer | Layer 3 — IP | Internet layer — IP |
| Data Link | Layer 2 — separate | Merged into Network Access (4-layer) |
| Physical | Layer 1 — separate | Merged into Network Access (4-layer) |
| Protocol Dependency | Protocol-independent (generic model) | Protocol-specific (built around TCP/IP) |
| Usage | Reference model, teaching | Actual Internet implementation |
| Approach | Top-down design | Bottom-up (developed from existing protocols) |

## 2.4 Encapsulation & Decapsulation

**Encapsulation** is the process by which each layer adds its own header (and sometimes trailer) to the data received from the layer above, creating a new PDU. **Decapsulation** is the reverse at the receiver.

```
SENDER (Encapsulation — headers added top-down): Application → Data (HTTP payload)
Transport → [TCP Header | Data] (Segment) Network → [IP Header | TCP Hdr | Data]
(Packet) Data Link → [ETH Hdr | IP Hdr | TCP | Data | FCS] (Frame) Physical → Bits
transmitted over the wire RECEIVER (Decapsulation — headers stripped bottom-up):
Physical → Data Link strips ETH header → Network strips IP header → Transport strips
TCP header → Application receives Data
```

### ■ INTERVIEW TIP — PDU Names by Layer

Memorise: Physical=Bit, Data Link=Frame, Network=Packet, Transport=Segment, Application=Data/Message. Interview question: 'What is a packet?' — technically a Network-layer PDU, not a generic term for any data unit.

### ■ QUICK REVISION BOX

- ◆ OSI mnemonic: 'Please Do Not Throw Sausage Pizza Away' (Physical→Application)
- ◆ TCP/IP collapses OSI layers 5+6+7 → Application; 1+2 → Network Access
- ◆ PDUs: Bit(L1) Frame(L2) Packet(L3) Segment(L4) Data(L5-7)
- ◆ Encapsulation = headers added by each layer on sender side
- ◆ OSI = reference/teaching model; TCP/IP = what the Internet actually uses
- ◆ Switch operates at L2 (MAC); Router at L3 (IP); Gateway at L4-L7

# CHAPTER 3: PHYSICAL LAYER

## 3.1 Signals & Transmission Media

The Physical layer converts bits into **signals** for transmission. An **analog signal** is continuous and varies smoothly (sine wave); a **digital signal** has discrete levels (0 or 1). Periodic signals repeat their pattern; aperiodic signals do not.

Key distinction: **Bandwidth** in the analog domain = range of frequencies (Hz); in the digital domain = maximum data rate (bps). **Baud rate** = number of signal changes per second (symbols/sec). Baud rate × bits-per-symbol = bit rate.

### *Guided (Wired) Media*

| Medium | Type | Speed / Distance | Notes |
|---|---|---|---|
| UTP Cat5e | Twisted Pair | 1 Gbps / 100m | RJ-45; susceptible to EMI; cheap |
| UTP Cat6/6a | Twisted Pair | 10 Gbps / 55m (6a: 100m) | Improved shielding; common in modern LANs |
| STP | Shielded Twisted Pair | 10 Gbps / 100m | Metal foil shield; better EMI resistance; expensive |
| Coaxial | Coaxial Cable | 10 Mbps–10 Gbps | TV cable, DOCSIS; single inner conductor |
| Single-mode Fiber | Fiber Optic | 100 Gbps+ / 100 km+ | Thin core; laser source; long-haul WAN |
| Multi-mode Fiber | Fiber Optic | 10–100 Gbps / 550m | Larger core; LED source; data center / campus |

### *Unguided (Wireless) Media*

**Radio Waves**: Omnidirectional; 3 kHz – 1 GHz; used for AM/FM, cellular (LTE/5G), Bluetooth.

**Microwaves**: Directional; 1–300 GHz; line-of-sight; used for satellite links, point-to-point backbone, Wi-Fi (2.4/5/6 GHz).

**Infrared**: Very short range; blocked by walls; used for TV remotes, IrDA.

**Satellite**: Geostationary (GEO, 35,786 km, ~270 ms delay), MEO, LEO (Starlink, ~20 ms).

## 3.2 Modulation, Multiplexing & Line Coding

### *Modulation Techniques*

| Technique | What Varies | Use Case |
|---|---|---|
| AM (Amplitude Modulation) | Carrier amplitude | AM radio; simple but noise-prone |
| FM (Frequency Modulation) | Carrier frequency | FM radio; better noise immunity |
| PM (Phase Modulation) | Carrier phase | Basis for digital PSK |
| ASK (Amplitude Shift Keying) | Amplitude (digital) | Simple; used in optical fiber |
| FSK (Frequency Shift Keying) | Frequency (digital) | Modems, RFID |
| PSK (Phase Shift Keying) | Phase (digital) | Wi-Fi, Bluetooth (BPSK, QPSK) |
| QAM (Quadrature AM) | Amplitude + Phase | Cable modem, DSL, Wi-Fi (64-QAM, 256-QAM, 1024-QAM) |

### *Multiplexing*

**FDM (Frequency Division)**: Divide the spectrum into sub-bands; each user gets a band. Analog; used in cable TV, AM/FM radio.

**TDM (Time Division)**: Divide time into slots; each user gets a slot in round-robin. Synchronous TDM wastes slots; Statistical TDM (STDM) is dynamic.

**WDM (Wavelength Division)**: FDM for fiber optics — different wavelengths (colors of light) on the same fiber. DWDM supports 100+ channels.

**CDM (Code Division / CDMA)**: Each user gets a unique orthogonal spreading code; all transmit simultaneously on the same frequency. Used in 3G cellular.

### *Line Coding (Digital-to-Digital)*

```
NRZ-L: 1=High, 0=Low [no transitions within bit; DC component problem] NRZ-I: Invert on 1, no change on 0 [solves long 0 runs] Manchester: 1=High→Low, 0=Low→High [always transitions; used in 10BASE-T Ethernet] Diff. Man.: Transition at START of 0 bit; no transition at start of 1 bit [token ring] 4B/5B: Block coding; every 4 bits mapped to 5-bit code with ≤3 trailing zeros
```

## 3.3 Shannon Capacity & Nyquist Theorem

**Nyquist Theorem** (for noiseless channel): Maximum Data Rate = 2 × Bandwidth × log■(L), where L = number of signal levels. Example: 3 kHz channel, 8 signal levels → 2 × 3000 × log■(8) = 18,000 bps.

**Shannon's Theorem** (for noisy channel): Channel Capacity C = B × log■(1 + SNR), where B = bandwidth (Hz) and SNR = signal-to-noise ratio. This is the theoretical maximum — regardless of signal levels used. Example: B = 1 MHz, SNR = 1023 → C = 10■ × log■(1024) = 10 Mbps.

**■ INTERVIEW TIP — Nyquist vs Shannon**

Nyquist is for noiseless channels — uses signal levels. Shannon is for noisy channels — uses SNR. In practice, the actual achievable data rate is the minimum of both. SNR in dB: SNR_dB = 10 log■■(SNR).

**■ QUICK REVISION BOX**

- ◆ Baud rate ≠ bit rate; bit rate = baud × bits-per-symbol
- ◆ Single-mode fiber: laser, long range (100+ km); Multi-mode: LED, short range (550m)
- ◆ FDM divides by frequency; TDM by time; WDM by wavelength (fiber); CDMA by code
- ◆ Manchester encoding always has a mid-bit transition — self-clocking
- ◆ Shannon: C = B·log■(1+SNR) — sets absolute theoretical limit regardless of signal levels
- ◆ Nyquist: Max rate = 2B·log■(L) — for noiseless channel

# CHAPTER 4: DATA LINK LAYER

## 4.1 Framing & Error Detection/Correction

The Data Link layer wraps packets into **frames** — adding source/destination MAC addresses, a length/type field, and a trailer for error detection (FCS/CRC). Framing methods include:

**Character Count**: First field indicates number of characters in frame. Fragile — error in count corrupts synchronisation.

**Byte Stuffing**: Special flag byte (e.g., 0x7E) marks frame boundaries. Escape byte (0x7D) inserted before any data byte that matches flag.

**Bit Stuffing**: HDLC: flag = 01111110. After 5 consecutive 1s in data, a 0 is stuffed. Receiver de-stuffs. Transparent to data content.

### *Error Detection — CRC (Cyclic Redundancy Check)*

**CRC** treats the data as a polynomial. The sender divides the data polynomial by a pre-agreed generator polynomial (e.g., CRC-32 generator used in Ethernet) using binary (modulo-2) division. The remainder (CRC) is appended to the frame. The receiver performs the same division — if remainder is 0, no error.

```
CRC Example (simplified CRC-4): Data: 10110010 (polynomial: x^7 + x^5 + x^4 + x^1)
Generator: 11001 (polynomial: x^4 + x^3 + 1; degree = 4, so append 4 zeros) Augmented:
101100100000 Divide by 11001 using modulo-2 (XOR) division: 101100100000 ÷ 11001 →
Remainder = 1110 Transmitted: 10110010 1110 Receiver divides entire 12 bits by 11001 →
remainder 0 = no error detected
```

**Parity Check**: Single-bit parity: add 1 bit so total 1s is even (even parity) or odd. Detects single-bit errors only, cannot correct.

**2D Parity (LRC)**: Arrange data in rows and columns; add parity bit for each row and column. Detects and can correct single-bit errors.

**Checksum**: Sum all data words; send 1s complement. Receiver sums all including checksum — should be all 1s. Used in IP, TCP, UDP.

### *Error Correction — Hamming Code*

**Hamming Code** adds redundant parity bits at positions that are powers of 2 (1, 2, 4, 8, ...) allowing single-bit error detection AND correction. For m data bits, r parity bits are needed such that 2^r ≥ m + r + 1.

```
Hamming Code Example — Data = 1011 (4 bits, need r=3 parity bits → 7-bit codeword):
Positions: 7 6 5 4 3 2 1 Data bits: d4 d3 d2 p4 d1 p2 p1 Insert data: 1 0 1 ? 1 ? ? p1
(pos 1) covers pos 1,3,5,7 → XOR(d1,d2,d4)=XOR(1,1,1)=1 → p1=1 p2 (pos 2) covers pos
2,3,6,7 → XOR(d1,d3,d4)=XOR(1,0,1)=0 → p2=0 p4 (pos 4) covers pos 4,5,6,7 →
XOR(d2,d3,d4)=XOR(1,0,1)=0 → p4=0 Final codeword: 1 0 1 0 1 0 1 Error detection: if bit
3 flips, parity checks at positions using XOR indicate position 3
```

## 4.2 Flow Control Protocols

### *Stop-and-Wait ARQ*

The simplest protocol: sender transmits one frame and waits for an ACK before sending the next. Simple but inefficient — the link is idle during propagation delay + ACK time. Efficiency η = T_t ÷ (T_t + 2T_p) where T_t = transmission time, T_p = propagation delay. For high-bandwidth, long-delay links (satellite), efficiency can drop below 1%.

```
Stop-and-Wait Timeline: Sender: [Frame 0]----propagate----> [Process] Sender:
<----propagate----[ACK 0] [idle.........] <- wasted bandwidth! Sender: [Frame 1]---->
```

### *Sliding Window: Go-Back-N & Selective Repeat*

Sliding Window allows multiple unacknowledged frames in transit (window size W). If W ≥ 2T_p/T_t + 1, the link is kept 100% utilised. Sequence numbers must be modulo 2^n where n = number of sequence number bits.

| Feature | Go-Back-N (GBN) | Selective Repeat (SR) |
|---|---|---|
| Window Size (max) | 2^n – 1 (sender); 1 (receiver) | 2^(n–1) (both sender and receiver) |
| On Error | Retransmit erroneous frame + ALL subsequent | Retransmit ONLY the erroneous frame |
| Receiver Buffer | No buffering needed (discard out-of-order) | Buffer needed for out-of-order frames |
| Efficiency | Lower for high error rates | Higher; wastes less bandwidth |
| Complexity | Simpler | More complex (receiver must reorder) |
| Use case | Low error rate links | High error rate / satellite links |

## 4.3 MAC Protocols

### *Random Access Protocols*

**Pure ALOHA**: Transmit whenever data is ready. Retransmit after random backoff on collision. Maximum throughput ≈ 18.4% (1/2e).

**Slotted ALOHA**: Divide time into slots; transmit only at slot boundaries. Max throughput ≈ 36.8% (1/e) — twice pure ALOHA.

**CSMA (Carrier Sense Multiple Access)**: Listen before transmitting. 1-persistent: if busy wait then immediately retransmit when idle. Non-persistent: wait random time then sense again. p-persistent: when idle, transmit with probability p.

**CSMA/CD (Collision Detection)**: Used in Ethernet (IEEE 802.3). Sense before sending. If collision detected during transmission, abort, send jam signal, wait random backoff (binary exponential backoff), retry.

**CSMA/CA (Collision Avoidance)**: Used in Wi-Fi (IEEE 802.11). Cannot detect collision during transmission in wireless. Uses DIFS + random backoff + optional RTS/CTS exchange to avoid collisions.

### *Controlled Access & Channelization*

**Polling**: Central controller polls each device in turn. No collision; deterministic. Controller = single point of failure.

**Token Passing**: A token circulates; only the holder can transmit. Used in Token Ring (IEEE 802.5), FDDI.

**FDMA/TDMA/CDMA**: Channel divided by frequency/time/code. Applicable to wireless and wired channelization (see Physical Layer).

## 4.4 ARP, Ethernet, VLAN & STP

### *MAC Address & ARP*

A **MAC (Media Access Control) address** is a 48-bit (6-byte) hardware address burned into the NIC. Written as 6 colon-separated hex pairs (e.g., AA:BB:CC:DD:EE:FF). The first 3 bytes = OUI (Organisationally Unique Identifier, assigned to vendor); last 3 = device-specific. Broadcast MAC = FF:FF:FF:FF:FF:FF.

**ARP (Address Resolution Protocol)** resolves a known IP address to an unknown MAC address within a LAN. Process: (1) Host A wants to send to IP 192.168.1.5. A checks its ARP cache — not found. (2) A broadcasts an ARP Request: 'Who has 192.168.1.5? Tell 192.168.1.1' (3) Host B (192.168.1.5) unicasts an ARP Reply with its MAC. (4) A updates its ARP cache (TTL typically 20 min). ARP cache can be poisoned — a security risk (ARP spoofing / ARP poisoning).

### Ethernet (IEEE 802.3) Frame Format

```
+----------+----------+--------+------+-------+-----+ | Preamble | Dest MAC | Src MAC|
Type | Data | FCS | | 8 bytes | 6 bytes | 6 bytes|2 bytes|46-1500|4 bytes|
+----------+----------+--------+------+-------+-----+ Preamble: 7 bytes 10101010 + 1
byte 10101011 (SFD) — synchronisation Type/Length: >1500 = EtherType (e.g., 0x0800=IPv4,
0x0806=ARP, 0x86DD=IPv6) FCS: 4-byte CRC-32 — Frame Check Sequence for error detection
```

### VLAN (Virtual LAN)

A **VLAN** logically segments a physical switch into multiple broadcast domains. Devices in different VLANs cannot communicate directly — they need a Layer 3 router or Layer 3 switch. IEEE 802.1Q adds a 4-byte tag (TPID + Priority + CFI + 12-bit VLAN ID, supporting 4094 VLANs) to the Ethernet frame. Benefits: security isolation, reduced broadcast traffic, flexible logical grouping.

### STP (Spanning Tree Protocol — IEEE 802.1D)

**STP** prevents Layer 2 loops by logically blocking redundant links while maintaining backup paths. Process: (1) Elect a **Root Bridge** — switch with lowest Bridge ID (Priority + MAC). (2) Each non-root switch finds its **Root Port** — lowest cost path to root. (3) Each segment elects a **Designated Port** — lowest cost to root on that segment. (4) All other ports become **Blocked**. BPDUs (Bridge Protocol Data Units) are exchanged. RSTP (802.1w) is the modern faster version (seconds vs ~50 sec for STP).

#### ■ QUICK REVISION BOX

- ◆ CRC appends remainder of data÷generator polynomial; receiver divides — zero remainder means no error
- ◆ Hamming Code: parity bits at positions 2^k; can detect AND correct 1-bit errors
- ◆ Go-Back-N: window = 2^n–1, retransmit all; Selective Repeat: window = 2^(n-1), retransmit only error
- ◆ CSMA/CD = Ethernet (detects collision); CSMA/CA = Wi-Fi (avoids collision, RTS/CTS)
- ◆ ARP: IP → MAC resolution via broadcast request + unicast reply; ARP cache stores mappings
- ◆ VLAN (802.1Q) creates logical broadcast domains; STP prevents L2 loops
- ◆ MAC = 48-bit hardware address; first 24 bits = OUI (vendor); Broadcast = FF:FF:FF:FF:FF:FF

# CHAPTER 5: NETWORK LAYER

## 5.1 IPv4 Header & Address Classes

The **Network Layer** provides logical (IP) addressing and routing across multiple networks. Its key functions are: logical addressing, routing (path determination), forwarding (moving packets), and fragmentation.

```
IPv4 HEADER (20 bytes minimum): +----+----+--------+------------------+ | Ver| IHL|
DSCP/ECN| Total Length | 0 +----+----+--------+------------------+ | Identification
|Flags| Fragment Off | 4 +-----------------+-----+--------------+ | TTL | Protocol |
Header Checksum | 8 +------+----------+---------------------+ | Source IP Address | 12
+-------------------------------------+ | Destination IP Address | 16
+-------------------------------------+ | Options (if IHL > 5) | 20+
+-------------------------------------+ Ver=4 (IPv4); IHL=Header length in 32-bit
words; TTL=hop limit; Protocol: 6=TCP, 17=UDP, 1=ICMP; Flags: DF(Don't Fragment),
MF(More Fragments)
```

### *IPv4 Address Classes*

| Class | First Bits | Range | Default Mask | Hosts/Net | Use |
|---|---|---|---|---|---|
| A | 0 | 1.0.0.0 – 126.255.255.255 | /8 (255.0.0.0) | 16,777,214 | Large orgs |
| B | 10 | 128.0.0.0 – 191.255.255.255 | /16 (255.255.0.0) | 65,534 | Medium orgs |
| C | 110 | 192.0.0.0 – 223.255.255.255 | /24 (255.255.255.0) | 254 | Small orgs/home |
| D | 1110 | 224.0.0.0 – 239.255.255.255 | N/A | N/A | Multicast |
| E | 1111 | 240.0.0.0 – 255.255.255.255 | N/A | N/A | Experimental |
| Special | - | 127.0.0.0/8 | - | - | Loopback (127.0.0.1) |
| Private A | - | 10.0.0.0 – 10.255.255.255 | /8 | ~16M | RFC 1918 Private |
| Private B | - | 172.16.0.0 – 172.31.255.255 | /12 | ~1M | RFC 1918 Private |
| Private C | - | 192.168.0.0 – 192.168.255.255 | /16 | 65534 | RFC 1918 Private |

## 5.2 Subnetting & VLSM — 5 Worked Examples

**Subnetting** divides a large network into smaller sub-networks. The subnet mask marks which bits are the network portion (1s) and which are the host portion (0s). CIDR notation (e.g., /24) counts the number of 1-bits in the mask. Key formulas: Number of subnets = 2^(borrowed bits); Hosts per subnet = 2^(host bits) – 2.

### *Example 1: 192.168.1.0/24 — divide into 4 equal subnets*

```
Borrow 2 bits from host portion (2^2=4 subnets). New prefix = /26 (255.255.255.192).
Subnet 0: 192.168.1.0/26 → Network: .0 Broadcast: .63 Hosts: .1–.62 (62 hosts) Subnet 1:
192.168.1.64/26 → Network: .64 Broadcast: .127 Hosts: .65–.126 Subnet 2:
192.168.1.128/26 → Network: .128 Broadcast: .191 Hosts: .129–.190 Subnet 3:
192.168.1.192/26 → Network: .192 Broadcast: .255 Hosts: .193–.254
```

### *Example 2: 10.0.0.0/8 — find network address, broadcast, hosts for 10.45.67.89/20*

```
Mask /20 = 255.255.240.0. Third octet 67 in binary = 0100 0011. Mask the 3rd octet: 240 =
1111 0000 → AND 0100 0011 = 0100 0000 = 64 Network: 10.45.64.0 Broadcast: 10.45.79.255
Hosts: 10.45.64.1–10.45.79.254 Usable hosts = 2^12 – 2 = 4094
```

### *Example 3: 172.16.0.0/16 — create subnets of at least 500 hosts each*

```
Need 500 hosts → 2^h – 2 ≥ 500 → h ≥ 9 bits → host bits = 9 → prefix = /23 Hosts per
subnet = 2^9 – 2 = 510. Subnets = 2^(23–16) = 2^7 = 128 subnets Subnet 0: 172.16.0.0/23
→ Broadcast 172.16.1.255, Hosts .0.1–.1.254 Subnet 1: 172.16.2.0/23 → Broadcast
172.16.3.255, Hosts .2.1–.3.254
```

### *Example 4: 192.168.10.130/27 — identify network, broadcast, first/last host*

```
Mask /27 = 255.255.255.224. Block size = 256−224 = 32. 130 ÷ 32 = 4 remainder 2 → belongs to block starting at 4×32=128 Network: 192.168.10.128 Broadcast: 192.168.10.159 First Host: .129 Last Host: .158 Usable: 30 hosts
```

### *Example 5 — VLSM: Assign 192.168.1.0/24 to: Dept A=100 hosts, B=50 hosts, C=25 hosts, D=10 hosts*

```
Sort largest first. VLSM allocates smallest needed block. A: need 100 hosts → /25 (126 hosts) → 192.168.1.0/25 (.0–.127) B: need 50 hosts → /26 (62 hosts) → 192.168.1.128/26 (.128–.191) C: need 25 hosts → /27 (30 hosts) → 192.168.1.192/27 (.192–.223) D: need 10 hosts → /28 (14 hosts) → 192.168.1.224/28 (.224–.239) Remaining: 192.168.1.240/28 available for future use
```

## 5.3 IPv6, NAT & ICMP

| Feature | IPv4 | IPv6 |
|---|---|---|
| Address Size | 32 bits (4 bytes) | 128 bits (16 bytes) |
| Address Space | ~4.3 billion | ~3.4 × 10^38 |
| Notation | Dotted decimal (192.168.1.1) | Hex colon (2001:db8::1) |
| Header Size | 20 bytes (variable) | 40 bytes (fixed, extensible) |
| Header Checksum | Yes (recomputed at each hop) | No (removed for speed) |
| Fragmentation | Routers and hosts | Only by source host |
| Broadcast | Yes | No; replaced by multicast/anycast |
| Configuration | Manual or DHCP | SLAAC (stateless) or DHCPv6 |
| Security | IPsec optional | IPsec mandatory (though rarely enforced) |
| NAT | Required (address exhaustion) | Not needed (huge address space) |

### *NAT (Network Address Translation)*

**Static NAT**: One-to-one mapping; each private IP maps to a fixed public IP. Used for servers that need a consistent public address.

**Dynamic NAT**: Pool of public IPs assigned dynamically to private IPs. No permanent mapping; not suitable for inbound connections.

**PAT / NAPT (Port Address Translation)**: Many private IPs map to ONE public IP using different port numbers. Most common form (home routers). Router maintains a translation table: (private IP, private port) ↔ (public IP, public port).

### *ICMP (Internet Control Message Protocol)*

ICMP is used for error reporting and network diagnostics. It is encapsulated in IP packets (Protocol = 1). Key message types:

| Type | Code | Meaning | Used By |
|---|---|---|---|
| 8 | 0 | Echo Request | ping |
| 0 | 0 | Echo Reply | ping response |
| 3 | 0–15 | Destination Unreachable (Net/Host/Port/Protocol) | Routers/hosts |
| 11 | 0 | Time Exceeded (TTL=0) | traceroute |
| 5 | 0–3 | Redirect (better route exists) | Routers |

**ping**: Sends ICMP Echo Request; measures RTT. **traceroute**: Sends packets with TTL=1,2,3,... Each router that decrements TTL to 0 returns an ICMP Time Exceeded. This reveals each hop's IP and RTT.

## 5.4 Routing Algorithms

### *Distance Vector vs Link State*

| Feature | Distance Vector (RIP) | Link State (OSPF) | Path Vector (BGP) |
|---|---|---|---|
| Algorithm | Bellman-Ford | Dijkstra's SPF | Path vector (custom) |
| Information Shared | Routing table to neighbours | Full topology (LSAs) to all | Path attributes to peers |
| Convergence | Slow (minutes) | Fast (seconds) | Slow but policy-driven |
| Scalability | Poor (max 15 hops for RIP) | Better (hierarchical areas) | Internet-scale (ASes) |
| Loop Prevention | Split horizon, poison reverse | No loops (complete topology) | AS-path attribute |
| Metric | Hop count | Cost (bandwidth-based) | BGP attributes (AS-path, MED, Local-Pref) |
| Protocol | RIP (UDP port 520) | OSPF (IP protocol 89) | BGP (TCP port 179) |
| Scope | IGP — small LANs | IGP — enterprise/ISP | EGP — between ISPs |

**Count-to-infinity problem** in Distance Vector: When a link fails, routers keep advertising incorrect routes through each other, incrementing hop count until infinity (15 for RIP). Solutions: split horizon (don't advertise route back through interface you learned it on), poison reverse (advertise metric=∞ back), hold-down timers.

## 5.5 Fragmentation & MPLS

When an IP packet is larger than the **MTU (Maximum Transmission Unit)** of a link (standard Ethernet MTU = 1500 bytes), it must be fragmented. Fields used: **Identification** — same for all fragments of a packet; **MF flag** — set on all fragments except the last; **Fragment Offset** — position of this fragment in the original datagram (units of 8 bytes). Reassembly happens only at the destination. In IPv6, only the source performs fragmentation; routers drop oversized packets and send an ICMPv6 'Packet Too Big' message.

**MPLS (Multiprotocol Label Switching)**: Adds a 32-bit label between L2 and L3 headers (sometimes called 'Layer 2.5'). Labels are assigned at the ingress Label Switch Router (LSR) based on FEC (Forwarding Equivalence Class). Each LSR just swaps labels — no IP lookup needed. Benefits: fast forwarding, traffic engineering, VPN support.

### ■ QUICK REVISION BOX

- ◆ IPv4 header is 20 bytes min; key fields: TTL (hop limit), Protocol (TCP=6, UDP=17, ICMP=1)
- ◆ Subnetting formula: subnets = 2^borrowed; hosts = 2^host_bits – 2
- ◆ CIDR /n means n bits are network; block size = 2^(32-n)
- ◆ VLSM: allocate largest subnet first, then carve remaining space
- ◆ PAT/NAPT: many private IPs → single public IP using different ports
- ◆ OSPF uses Dijkstra + LSAs; RIP uses Bellman-Ford + hop count ≤15; BGP = inter-AS path vector
- ◆ traceroute uses ICMP Time Exceeded by incrementing TTL from 1; each hop reveals itself

# CHAPTER 6: TRANSPORT LAYER

The **Transport Layer** provides logical **process-to-process** communication using **port numbers**. While the Network layer delivers packets between hosts (IP-to-IP), the Transport layer delivers segments between specific applications (port-to-port). It handles segmentation, reassembly, multiplexing (multiple applications sharing one IP), error control, and (for TCP) flow and congestion control.

**Well-Known Ports**: 0–1023: assigned by IANA. HTTP=80, HTTPS=443, FTP=21, SSH=22, DNS=53, SMTP=25, POP3=110, IMAP=143, DHCP=67/68, Telnet=23.

**Registered Ports**: 1024–49151: assigned to specific applications (e.g., MySQL=3306, PostgreSQL=5432, RDP=3389).

**Dynamic/Ephemeral Ports**: 49152–65535: used temporarily by clients as source ports.

## 6.1 TCP: Header, 3-Way & 4-Way Handshake

```
TCP HEADER (20 bytes minimum): +------------------+------------------+ | Source Port |
Destination Port | (16 + 16 bits) +------------------------------------ + | Sequence
Number | (32 bits) +---------------------------------------+ | Acknowledgement Number |
(32 bits) +----+-------+--+--+--+--+--+--+-------+ |Offs|
Reserv|NS|CW|EC|UR|AC|PS|RS|SY|FI| (4+4+8 flags)
+----+-------+-------------------------+ | Window Size | Checksum | (16 + 16)
+--------------------+------------------+ | Urgent Pointer | Options ... | (16 +
variable) +------------------+--------------------+ Flags: URG, ACK, PSH, RST, SYN, FIN
Window Size: advertised receive buffer (bytes) — used for flow control
```

### *TCP 3-Way Handshake (Connection Establishment)*

TCP requires a three-step handshake to synchronise sequence numbers and establish a connection before data transfer. This ensures both sides are ready and prevents old duplicate segments from being misinterpreted.

```
CLIENT SERVER |--[SYN, seq=x]--------------->| Client chooses ISN x; SYN flag set
|<-[SYN+ACK, seq=y, ack=x+1]--| Server chooses ISN y; ACK x+1 |--[ACK, seq=x+1,
ack=y+1]--->| Client ACKs server's SYN | | |====[ DATA TRANSFER ]=========| ISN =
Initial Sequence Number (randomly chosen to avoid old segment confusion)
```

### *TCP 4-Way Handshake (Connection Termination)*

```
CLIENT (active closer) SERVER |--[FIN, seq=u]------------->| Client done sending; sends
FIN |<-[ACK, ack=u+1]-----------| Server ACKs (client enters FIN-WAIT-2) | ...server
finishes... | |<-[FIN, seq=v]-------------| Server done; sends its FIN |--[ACK,
ack=v+1]---------->| Client ACKs (enters TIME-WAIT) TIME-WAIT: 2×MSL (Maximum Segment
Lifetime, typically 2 minutes) Purpose: ensure final ACK arrived; prevent old segments
from confusing new connection
```

## 6.2 TCP Flow Control & Congestion Control

### *Flow Control — Sliding Window*

The receiver advertises its **receive window (rwnd)** in the TCP header. The sender must not have more than rwnd bytes of unacknowledged data outstanding. This prevents the sender from overwhelming the receiver's buffer. **Zero Window**: if rwnd=0, sender stops and periodically sends 1-byte probe packets to check if window opens. **Silly Window Syndrome**: receiver advertises tiny windows; fixed by Nagle's algorithm (buffer small data until window is large enough or ACK received).

### *Congestion Control*

The sender maintains a **congestion window (cwnd)**. The effective window = min(cwnd, rwnd). TCP uses four mechanisms: Slow Start, Congestion Avoidance, Fast Retransmit, Fast Recovery.

```
TCP Congestion Control — cwnd growth: cwnd | | /\ Slow Start (exponential: cwnd doubles each RTT) | / \ |/ \___ Congestion Avoidance (linear: +1 MSS per RTT) | \ | X ← packet loss detected | /\ Timeout: ssthresh=cwnd/2, cwnd=1, restart Slow Start | / 3 dup ACKs: ssthresh=cwnd/2, cwnd=ssthresh (Fast Recovery→CA) +---------------> time ssthresh = Slow Start Threshold Phase 1 — Slow Start: cwnd starts at 1 MSS; doubles each RTT until ssthresh Phase 2 — Congestion Avoidance: cwnd increases by 1 MSS per RTT (linear) Phase 3 — Fast Retransmit: On 3 duplicate ACKs, retransmit without waiting for timeout Phase 4 — Fast Recovery: After fast retransmit, set ssthresh=cwnd/2, cwnd=ssthresh+3
```

| Algorithm | Loss Detection | cwnd Response |
|---|---|---|
| TCP Tahoe | Timeout OR 3 dup ACKs | cwnd=1 MSS; restart Slow Start |
| TCP Reno | Timeout: cwnd=1; 3 dup ACKs: cwnd=ssthresh (Fast Recovery) | Timeout severe; dup ACK mild |
| TCP New Reno | Same as Reno but handles partial ACKs during Fast Recovery | Improved recovery |
| CUBIC | Cubic growth function; default in Linux | Better for high-BDP networks |
| BBR | Model-based; estimates bottleneck BW + RTT; Google | Ignores loss as signal; maximises throughput |

## 6.3 UDP: Header & Use Cases

```
UDP HEADER (8 bytes only): +------------------+------------------+ | Source Port | Destination Port | (16 + 16 bits) +------------------+------------------+ | Length | Checksum | (16 + 16 bits) +------------------+------------------+ No sequence number, no ACK, no window, no connection state. Checksum is optional in IPv4, mandatory in IPv6.
```

UDP is used where speed matters more than reliability: DNS (single request-response), DHCP (bootstrap protocol), VoIP (Skype, WebRTC), online gaming, video streaming (QUIC/HTTP3 builds reliability on top of UDP), TFTP, SNMP, RADIUS.

## 6.4 TCP vs UDP — Comprehensive Comparison

| Feature | TCP | UDP |
|---|---|---|
| Connection | Connection-oriented (3-way handshake) | Connectionless |
| Reliability | Guaranteed delivery via ACKs + retransmission | No guarantee; best-effort |
| Ordering | Guaranteed in-order delivery | No ordering; may arrive out of order |
| Header Size | 20+ bytes | 8 bytes (fixed) |
| Speed | Slower (overhead) | Faster (low overhead) |
| Flow Control | Yes (receive window) | No |
| Congestion Control | Yes (cwnd) | No |
| Error Check | Checksum + retransmit | Checksum only |
| Statefulness | Stateful (connection maintained) | Stateless |
| Use Cases | HTTP, HTTPS, FTP, SSH, email | DNS, VoIP, streaming, gaming, DHCP |
| Protocol Overhead | High (ACKs, state, handshake) | Minimal |

**■ INTERVIEW TIP — Why not always use TCP?**

TCP's overhead (handshake, ACKs, retransmission, head-of-line blocking) introduces latency. Real-time apps (VoIP, gaming) cannot tolerate retransmissions — a retransmitted voice packet arrives too late to be useful. UDP lets applications implement their own partial reliability if needed (e.g., QUIC/HTTP3, RTP for VoIP).

## ■ QUICK REVISION BOX

◆ TCP = connection-oriented, reliable, ordered, flow/congestion control; UDP = connectionless, fast, no guarantee

◆ 3-way handshake: SYN → SYN-ACK → ACK (synchronises ISNs)

◆ 4-way termination: FIN→ACK→FIN→ACK; TIME-WAIT = 2×MSL prevents old segment confusion

◆ cwnd: Slow Start (exponential), Congestion Avoidance (linear), Fast Retransmit (3 dup ACKs), Fast Recovery

◆ TCP header = 20 bytes; UDP header = 8 bytes fixed

◆ Well-known ports: HTTP=80, HTTPS=443, SSH=22, DNS=53, FTP=21, SMTP=25

# CHAPTER 7: APPLICATION LAYER

## 7.1 DNS — Domain Name System

**DNS** translates human-readable domain names (e.g., www.google.com) into IP addresses. It is a distributed, hierarchical, globally scalable database. DNS uses UDP port 53 (small queries) and TCP port 53 (zone transfers, large responses).

```
DNS HIERARCHY: . (Root) / \ .com .org .net .edu .in (TLD servers) / \ google amazon (Authoritative Name Servers) / www mail api (Sub-domains)
```

### *DNS Resolution Process*

When you type www.example.com in a browser:

```
1. Browser checks its own DNS cache → not found 2. OS checks /etc/hosts and its DNS cache → not found 3. OS queries the configured Local DNS Resolver (Recursive Resolver) [usually your ISP's or 8.8.8.8 (Google) or 1.1.1.1 (Cloudflare)] 4. Local resolver asks Root Name Server → returns .com TLD server IP 5. Local resolver asks .com TLD server → returns example.com auth server IP 6. Local resolver asks example.com Authoritative NS → returns 93.184.216.34 7. Resolver returns IP to client; caches it with TTL Recursive query: client asks resolver to do all the work Iterative query: resolver does each step; root/TLD give referrals
```

| Record | Type | Purpose / Example |
|---|---|---|
| A | IPv4 Address | www.example.com → 93.184.216.34 |
| AAAA | IPv6 Address | www.example.com → 2606:2800:220:1:248:1893:25c8:1946 |
| CNAME | Alias | mail.example.com → mailserver.example.com (canonical name) |
| MX | Mail Exchanger | example.com → mail.example.com (for email routing) |
| NS | Name Server | example.com → ns1.dnshosting.com (authoritative server) |
| TXT | Text | SPF records, domain verification (Google, AWS), DKIM |
| PTR | Reverse DNS | 34.216.184.93.in-addr.arpa → www.example.com |
| SOA | Start of Authority | Zone serial, refresh, retry, expiry, min-TTL |

## 7.2 HTTP — HyperText Transfer Protocol

| Feature | HTTP/1.0 | HTTP/1.1 | HTTP/2 | HTTP/3 |
|---|---|---|---|---|
| Connection | New TCP per request | Persistent (Keep-Alive) | Persistent; multiplexed | QUIC (UDP-based) |
| Pipelining | No | Yes (but HOL blocking) | Multiplexing (no HOL) | No HOL (per stream) |
| Multiplexing | No | No | Yes (binary streams) | Yes (QUIC streams) |
| Compression | No | No (header) | HPACK (headers) | QPACK (headers) |
| Transport | TCP | TCP | TCP | QUIC/UDP |
| Server Push | No | No | Yes | Yes |
| TLS | Optional | Optional | Practically required | Required (TLS 1.3) |
| HOL Blocking | Yes | Yes | L7 solved; L4 still | Solved at L4 (QUIC streams) |

### HTTP Methods

| Method | Purpose | Safe? | Idempotent? | Body? |
|---|---|---|---|---|
| GET | Retrieve resource | Yes | Yes | No |
| POST | Create/submit data | No | No | Yes |
| PUT | Replace resource entirely | No | Yes | Yes |
| PATCH | Partial update | No | No* | Yes |
| DELETE | Delete resource | No | Yes | Optional |
| HEAD | GET without body (metadata) | Yes | Yes | No |
| OPTIONS | Discover allowed methods (CORS preflight) | Yes | Yes | No |
| TRACE | Echo request for debugging | Yes | Yes | No |

### HTTP Status Codes

| Code | Meaning | Use Case |
|---|---|---|
| 200 OK | Request succeeded | Standard success response |
| 201 Created | Resource created | POST to create new resource |
| 204 No Content | Success, no body | DELETE or update with no response |
| 301 Moved Permanently | Permanent redirect | Domain change; cached by browser |
| 302 Found | Temporary redirect | Load balancing, temporary URL change |
| 304 Not Modified | Cached version valid | Conditional GET (ETag/Last-Modified) |
| 400 Bad Request | Malformed syntax | Invalid JSON, missing required fields |
| 401 Unauthorized | Authentication required | Missing/invalid JWT or session |
| 403 Forbidden | Authenticated but no permission | RBAC denial; different from 401 |
| 404 Not Found | Resource not found | Wrong URL, deleted resource |
| 429 Too Many Requests | Rate limit exceeded | API throttling |
| 500 Internal Server Error | Server-side bug | Unhandled exception |
| 502 Bad Gateway | Upstream server error | Nginx can't reach app server |
| 503 Service Unavailable | Server down/overloaded | Maintenance, circuit breaker open |
| 504 Gateway Timeout | Upstream timeout | App server too slow |

## 7.3 HTTPS & TLS/SSL Handshake

HTTPS = HTTP over TLS (Transport Layer Security). TLS provides confidentiality (encryption), integrity (MAC/HMAC), and authentication (certificates). Port 443.

### TLS 1.2 Handshake

```
CLIENT SERVER |--[ClientHello]---------------->| TLS version, random, cipher suites,
extensions |<-[ServerHello]----------------| Chosen cipher suite, server random
|<-[Certificate]----------------| Server's X.509 certificate (public key)
|<-[ServerHelloDone]------------| |--[ClientKeyExchange]--------->| Pre-Master Secret
(encrypted with server's pub key) | [Both sides derive Master Secret from Pre-Master +
randoms] |--[ChangeCipherSpec + Finished]>| All future messages encrypted
|<-[ChangeCipherSpec + Finished]-| Handshake verified |====[ Encrypted HTTP Data ]====|
Total: 2 RTT before data (+ TCP handshake = 3 RTT from scratch)
```

### *TLS 1.3 Improvements*

**1-RTT Handshake**: Client sends key_share in first message; server responds with its key_share + Finished. Only 1 RTT needed (vs 2 in TLS 1.2).

**0-RTT Resumption**: If client has a pre-shared key from previous session, it can send data in the first message. Risk: replay attacks (non-idempotent requests).

**Removed Weak Algorithms**: TLS 1.3 removes RSA key exchange, MD5, SHA-1, DES, 3DES, RC4, export ciphers. Only strong AEAD ciphers allowed.

**Forward Secrecy**: Mandatory use of ephemeral Diffie-Hellman (DHE/ECDHE). Past sessions cannot be decrypted even if long-term key is compromised.

## 7.4 Other Application Layer Protocols

| Protocol | Port | Transport | Purpose |
|---|---|---|---|
| FTP (Active) | 20 (data), 21 (control) | TCP | File transfer; server opens data conn back to client |
| FTP (Passive) | 21 (control), >1023 (data) | TCP | Server opens high port; client connects; firewall-friendly |
| SMTP | 25 (server), 587 (submit) | TCP | Send email between mail servers |
| POP3 | 110 (plain), 995 (TLS) | TCP | Download email; delete from server; offline use |
| IMAP | 143 (plain), 993 (TLS) | TCP | Access email on server; sync across devices; modern choice |
| DHCP | 67 (server), 68 (client) | UDP | Dynamic IP address assignment via DORA |
| SNMP | 161 (agent), 162 (trap) | UDP | Network device management and monitoring |
| SSH | 22 | TCP | Secure remote shell; encrypted replacement for Telnet |
| Telnet | 23 | TCP | Unencrypted remote terminal; deprecated |

### *DHCP — DORA Process*

```
Client DHCP Server |--[DISCOVER]-->| Broadcast: 'Anyone have an IP for me?'
|<--[OFFER]----| Server offers: IP, mask, gateway, DNS, lease time |--[REQUEST]--->|
Client accepts offer (broadcast to inform all servers) |<--[ACK]------| Server confirms;
client configures its NIC DHCP Relay Agent: forwards DHCP broadcasts between subnets
(enables DHCP across routers)
```

### *WebSocket*

**WebSocket** (RFC 6455) enables full-duplex communication over a single, long-lived TCP connection. It starts with an HTTP/1.1 upgrade handshake (HTTP 101 Switching Protocols), then switches to the WebSocket frame format. Unlike HTTP polling, there is no per-message overhead after upgrade. Use cases: live chat, real-time dashboards, collaborative editors, online games. Compare to HTTP Long-Polling (client sends request; server holds until data available — wastes connections) and SSE (Server-Sent Events — server-to-client only).

**■ QUICK REVISION BOX**

- ◆ DNS hierarchy: Root → TLD (.com) → Authoritative NS → subdomain; uses UDP 53
- ◆ DNS record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (nameserver), TXT, PTR
- ◆ HTTP/2 multiplexes streams over 1 TCP; HTTP/3 uses QUIC/UDP to solve HOL blocking fully
- ◆ TLS 1.3 = 1-RTT handshake, forward secrecy mandatory, removed weak algorithms
- ◆ DHCP DORA: Discover → Offer → Request → Acknowledge
- ◆ 401=unauthenticated, 403=forbidden (authenticated but no permission), 404=not found, 502=bad gateway
- ◆ WebSocket upgrades HTTP to full-duplex TCP; suitable for real-time bidirectional apps

# CHAPTER 8: NETWORK SECURITY

## 8.1 CIA Triad & Cryptography

The **CIA Triad** defines the three core security goals: **Confidentiality** — only authorised parties can read data (encryption); **Integrity** — data is not altered in transit (hashing, MACs, digital signatures); **Availability** — services are accessible when needed (DDoS protection, redundancy). Additional goals: **Authentication** (verify identity), **Non-repudiation** (sender cannot deny sending — achieved via digital signatures).

| Type | Algorithm | Key Size | Speed | Use Case |
|---|---|---|---|---|
| Symmetric | AES-128/256 | 128/256 bits | Very fast | Bulk data encryption (HTTPS session data) |
| Symmetric | DES/3DES | 56/168 bits | Slow | Legacy; DES broken; 3DES deprecated |
| Asymmetric | RSA | 2048+ bits | Slow | Key exchange, digital signatures |
| Asymmetric | ECC (ECDSA) | 256 bits ≈ RSA 3072 | Fast | TLS, mobile devices |
| Asymmetric | Diffie-Hellman | Variable | Moderate | Key exchange (ECDHE in TLS 1.3) |
| Hash | MD5 | 128-bit output | Fast | Broken; not for security |
| Hash | SHA-1 | 160-bit output | Fast | Broken; not for security |
| Hash | SHA-256/512 | 256/512-bit output | Fast | TLS, digital signatures, blockchain |

**Digital Signature**: Sender hashes the message with SHA-256, then encrypts the hash with their **private** key → signature. Receiver decrypts with sender's **public** key, recomputes hash, compares — if equal: authentic and unmodified.

## 8.2 Firewalls, IDS/IPS & VPN

| Firewall Type | Layer | What It Inspects | Pros/Cons |
|---|---|---|---|
| Packet Filter | L3–L4 | IP addresses, ports, protocol | Fast, simple; no state tracking |
| Stateful Inspection | L3–L4 | Connection state table + packets | Tracks sessions; standard practice |
| Application Proxy | L7 | Full application payload | Deep inspection; high latency |
| NGFW (Next-Gen) | L3–L7 | DPI, identity, app awareness, IPS | Most capable; expensive |

**IDS (Intrusion Detection System)**: Passive — monitors traffic and alerts on suspicious patterns. Does NOT block traffic. Uses signature-based or anomaly-based detection.

**IPS (Intrusion Prevention System)**: Active — sits inline; can block/drop malicious traffic in real time. Signature + heuristic detection.

### *VPN (Virtual Private Network)*

VPN creates an encrypted tunnel over a public network, providing confidentiality, integrity, and authentication. Types: **Site-to-Site VPN** — connects two office networks; **Remote Access VPN** — individual users connect to corporate network. Key protocol: **IPsec** with two modes: **Transport Mode** (encrypts payload only, original IP header intact) and **Tunnel Mode** (encrypts entire original packet, adds new IP header — used for VPNs). IPsec protocols: **AH (Authentication Header)** — integrity + authentication, no encryption; **ESP (Encapsulating Security Payload)** — encryption + authentication (preferred).

### Common Attack Types

| Attack | Description | Prevention |
|---|---|---|
| SYN Flood (DoS) | Attacker sends millions of SYN packets without completing handshake; server exhausts state | SYN cookies, rate limiting, firewall |
| Man-in-the-Middle | Attacker intercepts and possibly modifies traffic between two parties | TLS/HTTPS, certificate pinning, HSTS |
| ARP Spoofing | Attacker sends fake ARP replies to associate their MAC with victim's IP; enables MITM on LAN | Dynamic ARP Inspection, static ARP entries |
| DNS Spoofing | Attacker provides forged DNS responses to redirect users to malicious IPs | DNSSEC, DoH/DoT |
| DDoS | Flood target with traffic from many sources (botnet); exhausts bandwidth or resources | CDN, Anycast, rate limiting, scrubbing centers |
| Session Hijacking | Attacker steals session token to impersonate a logged-in user | HTTPS, HttpOnly cookies, SameSite, short TTLs |

### Wireless Security

| Standard | Encryption | Security Level | Notes |
|---|---|---|---|
| WEP | RC4 (40-bit key) | Broken | Never use; cracked in minutes |
| WPA | TKIP (RC4 base) | Weak | Deprecated |
| WPA2 (Personal) | AES/CCMP (128-bit) | Good | Most common; vulnerable to PMKID, KRACK |
| WPA3 (Personal) | SAE (Dragonfly) | Strong | Replaces PSK; forward secrecy; 2018+ |

### ■ QUICK REVISION BOX

- ◆ CIA = Confidentiality, Integrity, Availability; also Authentication + Non-repudiation
- ◆ Symmetric (AES) is fast for bulk data; Asymmetric (RSA/ECC) for key exchange and signatures
- ◆ Digital Signature: hash with private key; verify with public key
- ◆ IPsec Tunnel Mode: encrypts entire original packet + adds new IP header — used in VPNs
- ◆ IDS = passive monitoring/alerting; IPS = active inline blocking
- ◆ WPA3/SAE replaces WPA2/PSK; provides forward secrecy via Dragonfly handshake
- ◆ SYN flood mitigated by SYN cookies; ARP spoofing by Dynamic ARP Inspection

# CHAPTER 9: WIRELESS & MOBILE NETWORKS

## 9.1 Wi-Fi (IEEE 802.11)

Wi-Fi uses **CSMA/CA** (Collision Avoidance) because wireless nodes cannot detect collisions during transmission (unlike wired Ethernet). Before transmitting, a station waits for DIFS (DCF Interframe Space), then a random backoff. Optional RTS/CTS handshake reduces the hidden node problem: A→AP: RTS; AP→all: CTS; now A transmits safely.

| Standard | Freq | Max Speed | MIMO | Wi-Fi Name |
|---|---|---|---|---|
| 802.11a | 5 GHz | 54 Mbps | No | — |
| 802.11b | 2.4 GHz | 11 Mbps | No | — |
| 802.11g | 2.4 GHz | 54 Mbps | No | — |
| 802.11n | 2.4 & 5 GHz | 600 Mbps | 4×4 MIMO | Wi-Fi 4 |
| 802.11ac | 5 GHz | 3.5 Gbps | 8×8 MU-MIMO | Wi-Fi 5 |
| 802.11ax | 2.4, 5, 6 GHz | 9.6 Gbps | MU-MIMO+OFDMA | Wi-Fi 6/6E |
| 802.11be | 2.4, 5, 6 GHz | 46 Gbps | 16×16 MIMO | Wi-Fi 7 |

**BSS (Basic Service Set)**: One Access Point + associated stations. BSSID = AP's MAC address.

**ESS (Extended Service Set)**: Multiple BSSs connected via Distribution System (DS); stations can roam. SSID = human-readable network name.

**Ad-hoc (IBSS)**: Devices communicate directly without an AP. Limited range and scalability.

## 9.2 Bluetooth & Cellular

**Bluetooth** (IEEE 802.15.1) operates at 2.4 GHz using frequency hopping spread spectrum (1600 hops/sec across 79 channels) to avoid interference. A **piconet** has 1 master + up to 7 active slaves. Multiple piconets sharing a slave device form a **scatternet**.

| Generation | Technology | Data Speed | Key Feature |
|---|---|---|---|
| 1G | AMPS (analog) | — | Voice only; no data; no security |
| 2G | GSM / CDMA (digital) | 9.6–50 kbps (GPRS 2.5G) | SMS, voice encryption |
| 3G | WCDMA / CDMA2000 | 2–14.4 Mbps (HSPA+) | Mobile internet, video calls |
| 4G / LTE | OFDMA, MIMO | 100 Mbps–1 Gbps | All-IP; VoLTE; low latency |
| 5G | mmWave, Massive MIMO | 10+ Gbps | <1ms latency; IoT; network slicing |

### ■ QUICK REVISION BOX

- ◆ CSMA/CA (Wi-Fi): sense before transmit + random backoff; RTS/CTS solves hidden node problem
- ◆ Wi-Fi 6 (802.11ax): OFDMA + MU-MIMO; serves many devices efficiently; operates on 6 GHz band (6E)
- ◆ BSS = single AP + stations; ESS = multiple BSS with roaming; BSSID = AP MAC; SSID = name
- ◆ Bluetooth piconet: 1 master + 7 active slaves; 2.4 GHz frequency hopping
- ◆ 5G delivers <1ms latency, 10+ Gbps; enables network slicing, massive IoT

# CHAPTER 10: ADVANCED & MODERN NETWORKING CONCEPTS

## 10.1 CDN, Load Balancing & Proxies

### *CDN (Content Delivery Network)*

A **CDN** is a globally distributed network of servers (Points of Presence / PoPs) that cache static content close to users. When a user requests content, DNS resolves to the nearest PoP rather than the origin server. Benefits: reduced latency (proximity), reduced origin load (caching), DDoS resilience (traffic absorbed across global network), higher availability. Examples: Cloudflare, AWS CloudFront, Akamai, Fastly.

### *Load Balancing*

**Layer 4 Load Balancing**: routes based on IP/port only; doesn't inspect content. Fast; used for TCP/UDP. **Layer 7 Load Balancing**: inspects HTTP headers, URL, cookies; can route based on path (/api vs /static), host header, or session.

| Algorithm | Description | Best For |
|---|---|---|
| Round Robin | Each server takes turns receiving requests | Uniform request sizes |
| Weighted Round Robin | Servers with higher weight get more requests | Heterogeneous server capacity |
| Least Connections | Route to server with fewest active connections | Long-lived connections (WebSocket) |
| IP Hash | Hash client IP to select server (sticky sessions) | Session affinity without cookies |
| Random | Random selection from pool | Simple; works for uniform loads |

### *Reverse Proxy vs Forward Proxy*

| Feature | Forward Proxy | Reverse Proxy |
|---|---|---|
| Sits between | Client and Internet | Internet and Server(s) |
| Client knows proxy? | Yes (configured) | No (transparent to client) |
| Purpose | Client anonymity, content filtering, caching, bandwidth | Load balancing, SSL termination, caching, security |
| Examples | Squid, corporate proxy | Nginx, HAProxy, Cloudflare |
| Hides | Client identity from server | Server identity/count from client |

## 10.2 Network Troubleshooting & Tools

| Tool | Layer | Purpose |
|---|---|---|
| ping | L3 (ICMP) | Test connectivity; measure RTT; check reachability |
| traceroute/tracert | L3 (ICMP/UDP) | Discover path hops and latency to destination |
| netstat / ss | L4 (Transport) | Show active connections, listening ports, socket state |
| tcpdump | L2-L7 | CLI packet capture; filter by IP, port, protocol |
| Wireshark | L2-L7 | GUI packet analyser; deep protocol dissection |
| nslookup / dig | L7 (DNS) | DNS query debugging; check A, MX, NS records; trace delegation |
| curl / wget | L7 (HTTP) | Test HTTP endpoints; download files; inspect headers (-v) |
| nmap | L3-L4 | Port scanning; OS detection; service version detection |
| telnet | L7 | Test TCP port connectivity (telnet host port); deprecated for terminal use |
| arp -a | L2-L3 | Show/manage ARP cache; diagnose ARP poisoning |
| ifconfig/ip addr | L2-L3 | Show/configure network interfaces, IP addresses, MAC |
| route/ip route | L3 | Show/modify routing table |

## 10.3 QoS & Modern Concepts

**QoS (Quality of Service)**: Mechanisms to prioritise certain traffic types. IntServ (per-flow reservation, RSVP) vs DiffServ (DSCP marking in IP header, per-hop behaviour). Traffic shaping = smooth out bursts (leaky bucket, token bucket); traffic policing = drop/mark packets exceeding rate.

**Unicast / Broadcast / Multicast / Anycast**: Unicast: one sender → one receiver. Broadcast: one → all on subnet. Multicast: one → group (IGMP, Class D). Anycast: one → nearest of multiple servers (used in CDNs and BGP for routing to nearest DNS server like 1.1.1.1).

**Zero Trust Architecture**: Security model: 'never trust, always verify.' No implicit trust based on network location. Every request authenticated and authorised, even inside the corporate network. Key technologies: identity-aware proxies, MFA, micro-segmentation.

**NAT Traversal (STUN/TURN/ICE)**: When two devices behind NAT need to connect directly (WebRTC, P2P). STUN discovers public IP/port. TURN relays traffic through a server when direct connection fails. ICE tries all candidate paths and picks the best.

### ■ QUICK REVISION BOX

- ◆ CDN = distributed PoPs cache static content near users; reduces latency + origin load
- ◆ Layer 4 LB = IP/port routing (fast); Layer 7 LB = HTTP-aware routing (flexible)
- ◆ Reverse proxy hides server from clients (Nginx, CloudFlare); Forward proxy hides clients from server
- ◆ ping=reachability; traceroute=path; tcpdump=packet capture; dig=DNS debug; nmap=port scan
- ◆ Anycast routes to nearest server — enables 1.1.1.1 DNS to work globally
- ◆ Zero Trust: authenticate everything; no implicit internal trust; micro-segment the network

# CHAPTER 11: TOP 50 COMPUTER NETWORKS INTERVIEW QUESTIONS

Each question below includes a detailed explanation and a one-sentence Quick Recall summary. These represent the most commonly asked questions in SDE and DevOps interviews at top product-based companies.

### Q1. What is a computer network? What are its goals?

A computer network is a collection of autonomous computing devices interconnected by links and configured to exchange data. The word 'autonomous' is essential — each device runs its own OS independently.

The four fundamental goals are: (1) Resource Sharing — hardware (printers, storage), software, and data are accessible from any node; (2) Communication — enables email, VoIP, messaging, and collaborative work; (3) Reliability — redundant paths ensure data delivery even if one link fails; (4) Cost Reduction — sharing expensive resources (e.g., laser printers, servers) reduces per-user cost.

Networks also provide scalability (add nodes without redesigning the network) and enable cloud computing, IoT, and distributed systems that underpin modern technology.

■ ***Quick Recall: A network = autonomous nodes + communication links; goals = resource sharing, communication, reliability, cost reduction.***

### Q2. Differentiate between LAN, MAN, and WAN.

LAN (Local Area Network) covers a small geographic area such as a home, office, or building — typically up to 1 km. It uses Ethernet (IEEE 802.3) or Wi-Fi (802.11) and offers high speed (1–100 Gbps). All devices share the same broadcast domain unless segmented with VLANs.

MAN (Metropolitan Area Network) covers a city or campus — 5 to 50 km. Technologies include SONET, WiMAX, and metro Ethernet rings. ISPs often use MANs to connect business subscribers within a city.

WAN (Wide Area Network) spans countries or continents. The Internet itself is the world's largest WAN. WANs use technologies like MPLS, SD-WAN, leased lines, and submarine fiber optic cables. WAN links are typically slower, higher latency, and more expensive per bit than LAN links.

Key distinction: LAN = owned by single organisation, high speed; WAN = provider-managed, lower speed, higher latency.

■ ***Quick Recall: LAN = 1 km/fast/private; MAN = 5–50 km/city; WAN = global/Internet/lower speed per cost.***

### Q3. What are the different network topologies? Compare them.

Bus topology uses a single shared cable. All devices tap into it. Advantages: cheap and simple. Drawbacks: a break in the cable takes down the entire network; all devices share one collision domain, limiting performance.

Star topology connects all devices to a central hub or switch. Modern Ethernet uses star topology with switches. Advantages: a failed cable only isolates that one device; easy to add/remove nodes. Drawback: the central switch is a single point of failure.

Ring topology connects devices in a circle. Token Ring (IEEE 802.5) used this. Each packet travels around the ring. A break can bring the network down unless dual-ring (FDDI) is used.

Mesh topology — full mesh connects every node to every other (n(n-1)/2 links). Highly fault tolerant; used in WANs and backbone networks. Partial mesh balances cost and redundancy.

Tree (hierarchical) topology: root switch → distribution switches → access switches. Scalable and organised; used in enterprise networks. Hybrid combines two or more topologies.

■ ***Quick Recall: Star (switch-centred) is most common for LANs; mesh for WAN backbone; tree for enterprise hierarchical design.***

### Q4. What is the difference between a Hub, Switch, and Router?

A Hub operates at Layer 1 (Physical). It is a multi-port repeater — it receives a signal and broadcasts it to all ports. All connected devices are in the same collision domain and the same broadcast domain. Hubs are obsolete in modern networks.

A Switch operates at Layer 2 (Data Link). It learns MAC addresses and forwards frames only to the destination port — creating separate collision domains per port. All ports on a switch still share the same broadcast domain (unless VLANs are configured). Switches dramatically improve performance over hubs.

A Router operates at Layer 3 (Network). It routes packets between different networks using IP addresses. Routers separate broadcast domains — a broadcast on one interface does not propagate to another. Routers use routing tables and protocols (OSPF, BGP) to determine the best path.

Rule of thumb: Hub = dumb repeater; Switch = smart L2 forwarder; Router = inter-network connector + broadcast domain separator.

■ ***Quick Recall: Hub: L1, one collision domain; Switch: L2, per-port collision domain; Router: L3, separates broadcast domains.***

### Q5. Explain the OSI model with all 7 layers.

The OSI (Open Systems Interconnection) model is a conceptual 7-layer framework for network communication.

Layer 7 Application: interfaces directly with user-facing software (HTTP, DNS, SMTP, FTP). Layer 6 Presentation: data translation, encryption (TLS), and compression. Layer 5 Session: manages dialog sessions, synchronisation, and checkpointing. Layer 4 Transport: process-to-process delivery via port numbers; TCP (reliable) and UDP (fast). Layer 3 Network: logical addressing (IP) and routing across multiple networks. Layer 2 Data Link: node-to-node hop (MAC addresses), framing, error detection (CRC), access control. Layer 1 Physical: raw bits as electrical, optical, or radio signals.

PDU names from bottom to top: Bit, Frame, Packet, Segment, Data. Mnemonic: 'Please Do Not Throw Sausage Pizza Away'.

■ ***Quick Recall: OSI: L1=Physical, L2=DataLink, L3=Network, L4=Transport, L5=Session, L6=Presentation, L7=Application.***

### Q6. Compare the OSI model and TCP/IP model.

The OSI model has 7 layers and was designed by ISO as a theoretical framework. The TCP/IP model has 4–5 layers and is what the actual Internet uses.

Mapping: OSI Layers 5+6+7 (Session, Presentation, Application) collapse into TCP/IP's single Application layer. OSI Layer 4 (Transport) = TCP/IP Layer 4 (TCP/UDP). OSI Layer 3 (Network) = TCP/IP Internet layer (IP). OSI Layers 1+2 (Physical + Data Link) = TCP/IP Network Access layer (in 4-layer model).

OSI is protocol-independent and prescriptive — useful for teaching and troubleshooting. TCP/IP is protocol-specific and descriptive — it was designed around existing protocols (TCP and IP). OSI's strict layer boundaries are rarely enforced in practice.

■ ***Quick Recall: OSI=7 layers (reference model); TCP/IP=4–5 layers (actual Internet); OSI Apps=3 layers vs TCP/IP=1 Application layer.***

### Q7. What is encapsulation and decapsulation?

Encapsulation is the process where each layer of the OSI/TCP/IP model adds its own control information (header and sometimes trailer) to the data received from the layer above, creating a new PDU (Protocol Data Unit).

Sender sequence: Application data → Transport adds TCP/UDP header (Segment) → Network adds IP header (Packet) → Data Link adds Ethernet header + CRC trailer (Frame) → Physical converts to bits/signals.

Decapsulation is the reverse at the receiver: each layer strips its own header as it passes data upward. The Physical layer converts signals back to bits → Data Link validates CRC and strips Ethernet header → Network strips IP header → Transport strips TCP header → Application reads the data.

This layered approach allows each layer to function independently. Changing the Physical medium (e.g., from Ethernet to Wi-Fi) doesn't affect the Network or Transport layers.

> ■ ***Quick Recall: Encapsulation = each layer adds header going down; decapsulation = each layer strips header going up.***

### Q8. What is the difference between circuit switching and packet switching?

Circuit switching establishes a dedicated end-to-end path before any data is transmitted. Resources (bandwidth) are reserved for the entire duration of the call. The classic example is the PSTN (Public Switched Telephone Network). Advantages: guaranteed QoS, fixed delay. Disadvantages: bandwidth wasted when idle, inefficient for bursty data.

Packet switching breaks data into packets, each routed independently through the network based on destination address. No path is reserved. Routers make independent forwarding decisions per packet. The Internet uses datagram-based packet switching.

Key advantage of packet switching: statistical multiplexing — many users share bandwidth; when one is idle, others can use the bandwidth. Disadvantages: variable delay, potential packet loss during congestion.

Virtual circuit switching is a hybrid: a path is established once (like circuit switching), but packets still share links with others (like packet switching). Used in ATM and Frame Relay.

> ■ ***Quick Recall: Circuit switching: dedicated path, constant delay, wasted BW; Packet switching: shared, efficient, variable delay.***

### Q9. Explain the Physical layer and its functions.

The Physical layer (Layer 1) is responsible for transmitting raw bits over a physical medium. It defines the electrical, mechanical, and procedural interfaces between the networking equipment and the transmission medium.

Key functions: bit representation (how 0s and 1s are encoded as signals — e.g., Manchester encoding), data rate (bits per second), synchronisation (ensuring sender and receiver have the same bit timing), transmission mode (simplex/half-duplex/full-duplex), physical topology, and connection type (RS-232, RJ-45, SC/LC fiber connectors).

Important distinction: bandwidth at this layer means range of frequencies (Hz), not bit rate. Baud rate (symbols/sec) may differ from bit rate if more than 2 signal levels are used (e.g., QAM-64 encodes 6 bits per symbol).

The Physical layer does NOT deal with meaning or addressing — it only moves bits. Error detection is handled by Layer 2.

■ ***Quick Recall: Physical layer: converts bits to signals; defines encoding, data rate, connectors, and transmission mode.***

### Q10. What is the difference between guided and unguided media?

Guided media are physical cables that direct the signal along a specific path. Examples: Twisted Pair (UTP/STP), Coaxial Cable, and Fiber Optic.

Twisted Pair (UTP Cat6) is the most common LAN cable — cheap, flexible, supports 10 Gbps at 55m. Fiber optic uses light pulses — immune to EMI, low attenuation, supports 100+ Gbps over 100+ km (single-mode). Coaxial has a central conductor surrounded by shielding — used for cable TV (DOCSIS).

Unguided media transmit signals through air or space without a physical cable. Examples: radio waves (cellular, Wi-Fi), microwaves (point-to-point links, satellite), infrared (TV remotes), and light (Li-Fi).

Key trade-offs: guided media offer higher speed and security; unguided media provide mobility and ease of deployment but are susceptible to interference and interception.

■ ***Quick Recall: Guided = physical cable (twisted pair, coax, fiber); Unguided = wireless (radio, microwave, infrared).***

### Q11. What is multiplexing? Explain FDM, TDM, and WDM.

Multiplexing is the technique of combining multiple signals onto one shared communication channel, maximising bandwidth utilisation.

FDM (Frequency Division Multiplexing): the available bandwidth is divided into non-overlapping frequency bands. Each user is assigned a band continuously. Used in AM/FM radio, cable TV, DSL (upstream and downstream on different frequency bands).

TDM (Time Division Multiplexing): time is divided into fixed slots, and each user is assigned a slot in round-robin. Synchronous TDM wastes slots when users have no data. Statistical TDM (STDM) assigns slots dynamically — used in DSL, T1/E1 lines.

WDM (Wavelength Division Multiplexing): multiple wavelengths (colors) of light transmitted simultaneously on a single fiber. DWDM (Dense WDM) can carry 80–160+ channels. Used in long-haul fiber networks. CDM/CDMA assigns unique orthogonal codes to users who all transmit simultaneously.

■ ***Quick Recall: FDM=frequency bands; TDM=time slots; WDM=wavelengths of light on fiber; CDMA=orthogonal codes.***

### Q12. What is the Data Link layer? What are its responsibilities?

The Data Link layer (Layer 2) provides node-to-node data transfer over a single network link. While the Network layer routes packets between networks, the Data Link layer handles the hop-by-hop transfer within a network.

Responsibilities: Framing — encapsulating Network-layer packets into frames with MAC headers and FCS trailer. Error Detection — using CRC (Cyclic Redundancy Check) to detect corrupted frames (Data Link does NOT correct errors in most cases — IP/TCP handle recovery). Flow Control — prevents a fast sender from overwhelming a slow receiver. Media Access Control — in shared media, determines which station may transmit (CSMA/CD, CSMA/CA, token passing).

Sublayers: LLC (Logical Link Control, IEEE 802.2) — provides an interface to the Network layer, handles error/flow control. MAC (Medium Access Control) — handles framing, MAC addressing, and medium access.

Devices at this layer: bridges (2-port) and switches (multi-port). They forward based on MAC addresses, not IP addresses.

■ ***Quick Recall: Data Link = hop-to-hop delivery, framing, MAC addressing, CRC error detection, media access control.***

### Q13. Explain framing and its methods.

Framing is the Data Link layer's method of packaging data bits into a structured unit (frame) with defined start and end boundaries, so the receiver can identify frame limits.

Character Count: the first field specifies how many characters are in the frame. Very fragile — if this count field has a bit error, framing is lost for all subsequent frames.

Byte Stuffing (Character Stuffing): a special byte (flag byte, e.g., 0x7E in PPP) marks frame boundaries. If this byte appears in data, an escape byte (0x7D) is inserted before it. The receiver strips the escape byte. Doubles the flag byte if it appears in data.

Bit Stuffing: used in HDLC. The flag pattern is 01111110. If the data contains 5 consecutive 1s, the sender stuffs a 0 after them. The receiver removes any 0 following 5 consecutive 1s. This allows arbitrary data patterns.

Physical Layer Violations: certain signal patterns are illegal in the encoding scheme (e.g., a non-transition period in Manchester encoding). These illegal signals are used as frame delimiters — efficient but encoding-specific.

■ ***Quick Recall: Framing methods: character count, byte stuffing (escape byte), bit stuffing (insert 0 after 5×1s), physical violations.***

### Q14. What is CRC? How does error detection work?

CRC (Cyclic Redundancy Check) is a powerful error detection technique. The data is treated as a polynomial, and polynomial division (using modulo-2 arithmetic, which is XOR) is performed with a pre-agreed generator polynomial.

How it works: (1) Sender appends r zeros to the data (where r = degree of generator polynomial). (2) Divides augmented data by generator using modulo-2 division. (3) The remainder (r bits) is the CRC, which replaces the appended zeros. (4) The complete frame (data + CRC) is transmitted. (5) Receiver divides the entire received frame by the same generator. If the remainder is 0, the frame is error-free (with very high probability).

CRC-32 (used in Ethernet) uses a 32-bit generator polynomial and can detect all single-bit errors, all double-bit errors, all odd-number bit errors, and burst errors up to 32 bits in length. The probability of a 32-bit error going undetected is 2^-32.

CRC is far more reliable than simple parity. It cannot correct errors — only detect them. For error correction, Hamming codes or ARQ (retransmission) protocols are used.

■ ***Quick Recall: CRC: append r zeros, divide by generator polynomial (XOR/mod-2), send remainder as FCS; receiver divides, zero=no error.***

### Q15. What is Hamming Code? How does error correction work?

Hamming Code is a linear error-correcting code that can detect and correct single-bit errors (and detect double-bit errors with an extended version). Invented by Richard Hamming at Bell Labs in 1950.

Concept: for m data bits, we need r parity bits such that $2^r \geq m + r + 1$. The parity bits are placed at positions that are powers of 2 (1, 2, 4, 8, ...). Each parity bit covers a specific subset of bit positions (those whose binary representation has a 1 in the same position as the parity bit's position).

Error detection: The receiver recalculates all parity bits. If any parity bits are wrong, the binary value formed by all incorrect parity positions gives the exact position of the erroneous bit. That bit is then flipped to correct the error. This is why Hamming is called a 'self-correcting' code.

Example for 4 data bits: r = 3 parity bits needed (since 2^3=8 ≥ 4+3+1=8). Total codeword = 7 bits. Bit positions: 1(p1), 2(p2), 3(d1), 4(p4), 5(d2), 6(d3), 7(d4).

■ ***Quick Recall: Hamming Code: parity bits at positions 2^k; check all parity after receiving; binary position of failed checks = error position.***

### Q16. Explain Stop-and-Wait ARQ and its efficiency.

Stop-and-Wait ARQ (Automatic Repeat reQuest) is the simplest reliable data transfer protocol. The sender transmits one frame, then stops and waits for an ACK from the receiver before sending the next frame.

Error handling: if the ACK is not received within a timeout period, the sender retransmits the frame. To avoid duplicate frames (in case ACK is lost), frames are numbered with alternating 0 and 1 (1-bit sequence number).

Efficiency: the link is idle during the round-trip propagation time. Efficiency $\eta = T_t / (T_t + 2T_p + T_{proc}) \approx T_t / (T_t + 2T_p)$. For a 1 Mbps link with 1 KB frames and 250 ms propagation: $T_t = 8ms$, $2T_p = 500ms \rightarrow \eta = 8/508 \approx 1.6\%$. Extremely wasteful for high-bandwidth, long-delay links.

The parameter $a = T_p/T_t$ characterises the link. When $a >> 1$ (satellite links), Stop-and-Wait is catastrophically inefficient. The solution is to use sliding window protocols (Go-Back-N or Selective Repeat) to keep multiple frames in flight.

■ ***Quick Recall: Stop-and-Wait: send 1 frame, wait for ACK; efficiency = $T_t/(T_t + 2T_p)$; terrible for high-BDP links.***

### Q17. Explain Sliding Window protocol. Differentiate Go-Back-N and Selective Repeat.

Sliding Window allows the sender to have up to W frames (window size) unacknowledged simultaneously. This keeps the link busy. For 100% utilisation: $W \geq 1 + 2a$ where $a = T_p/T_t$.

Go-Back-N (GBN): sender window = 2^n - 1; receiver window = 1. The receiver accepts only in-order frames — any out-of-order frame is discarded. When an error occurs, the sender retransmits the erroneous frame and ALL subsequent frames. Simple receiver, but wasteful on high-error links.

Selective Repeat (SR): sender window = receiver window = 2^(n-1). The receiver buffers out-of-order frames. When an error occurs, ONLY the erroneous frame is retransmitted. Much more efficient on noisy links, but requires more receiver-side buffering and logic.

Window size constraint: GBN uses 2^n - 1 because if window = 2^n, the receiver cannot distinguish a new window from a retransmitted old window after wraparound. SR requires 2^(n-1) for the same reason, but stricter due to out-of-order buffering.

■ ***Quick Recall: GBN: window=2^n−1, retransmit all from error; SR: window=2^(n-1), retransmit only error, buffer out-of-order.***

### Q18. What is CSMA/CD? How does Ethernet use it?

CSMA/CD (Carrier Sense Multiple Access with Collision Detection) is the MAC protocol used by classic Ethernet (IEEE 802.3). It operates on shared media where multiple devices compete for the channel.

Protocol steps: (1) Sense: before transmitting, the device senses the channel. If busy, wait. (2) Transmit: if idle, begin transmission. (3) Monitor: while transmitting, continue to monitor the channel. (4) Collision: if a collision is detected (signal voltage exceeds normal), abort transmission and send a 32-bit jam signal to ensure all nodes know about the collision. (5) Backoff: wait a random time using binary exponential backoff — for the k-th collision, choose a random integer from [0, 2^k - 1] and wait that many slot times. (6) Retry.

Why monitor while transmitting? A collision occurs when two stations start transmitting within one propagation delay of each other. The colliding frames mix and corrupt. Without monitoring, the entire frame would be wasted. The minimum Ethernet frame size (64 bytes) ensures that transmission time ≥ 2 × propagation delay, guaranteeing that the sender will still be transmitting when the collision signal arrives.

Modern switches eliminate collisions entirely by providing a dedicated full-duplex link per device. CSMA/CD is still in the standard but irrelevant in practice today.

■ ***Quick Recall: CSMA/CD: sense→transmit→detect collision→jam→binary exponential backoff; modern switches eliminate collisions.***

### Q19. What is CSMA/CA? Where is it used?

CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) is the MAC protocol for Wi-Fi (IEEE 802.11). Unlike wired Ethernet, wireless stations cannot detect a collision while transmitting — by the time you detect a problem, the frame is already corrupted. Therefore, Wi-Fi avoids collisions proactively.

Protocol: (1) If channel is idle for DIFS (Distributed IFS) duration, a station may transmit. (2) If channel is busy, the station waits until it's idle, then waits DIFS, then waits a random backoff interval (chosen from [0, CW-1] where CW = contention window). (3) During backoff, if channel goes busy, the counter freezes. It resumes when channel is idle again (plus DIFS). This ensures collision avoidance.

RTS/CTS (Request to Send / Clear to Send): optional mechanism to solve the hidden node problem (two stations that can't hear each other but both transmit to the same AP, causing collision at the AP). Station sends RTS to AP → AP broadcasts CTS → other stations defer transmission during NAV (Network Allocation Vector) period.

ACK required: Wi-Fi acknowledges every unicast frame (unlike wired Ethernet which relies on TCP). If no ACK, frame is retransmitted.

■ ***Quick Recall: CSMA/CA: DIFS+random backoff to avoid collisions; RTS/CTS solves hidden node problem; ACK every unicast frame.***

### Q20. What is a MAC address? How is it different from an IP address?

A MAC (Media Access Control) address is a 48-bit (6-byte) hardware identifier permanently assigned (burned in) to a NIC by the manufacturer. It is written as 6 pairs of hexadecimal digits (e.g., AA:BB:CC:DD:EE:FF or AA-BB-CC-DD-EE-FF). The first 3 bytes are the OUI (Organisationally Unique Identifier) assigned by IEEE to the manufacturer; the last 3 bytes are device-specific.

IP addresses are logical, 32-bit (IPv4) or 128-bit (IPv6) addresses assigned by network administrators or DHCP. They are hierarchical (network part + host part) and can change (DHCP lease, network move).

Key differences: MAC is a physical, flat, Layer 2 identifier — globally unique per NIC, doesn't change based on network. IP is a logical, hierarchical, Layer 3 identifier — changes when you move to a different network. MAC is used within a LAN for hop-to-hop delivery. IP is used for end-to-end delivery across networks. ARP maps IP to MAC within a subnet.

■ ***Quick Recall: MAC=48-bit L2 hardware ID (vendor OUI + device); IP=32-bit L3 logical address, hierarchical, changes per network.***

### Q21. How does ARP work?

ARP (Address Resolution Protocol) resolves an IP address to a MAC address within the same subnet. This is necessary because even though IP provides end-to-end addressing, the actual frame transmission uses MAC addresses.

Process: (1) Host A wants to send a packet to 192.168.1.5. A checks its ARP cache — no entry found. (2) A broadcasts an ARP Request to FF:FF:FF:FF:FF:FF: 'Who has 192.168.1.5? Tell 192.168.1.1 (my IP), at AA:11:22:33:44:55 (my MAC).' (3) All devices on the subnet receive this broadcast. Only 192.168.1.5 (Host B) responds with a unicast ARP Reply: 'I am 192.168.1.5, my MAC is BB:22:33:44:55:66.' (4) Host A updates its ARP cache with the mapping (TTL ~ 20 minutes).

ARP Cache: Every device maintains a table of IP-to-MAC mappings to avoid repeated broadcasts. Gratuitous ARP: a device announces its own IP-MAC mapping (used when an IP address is configured or changed). ARP Spoofing: an attacker sends fake ARP replies to associate their MAC with another device's IP — enabling man-in-the-middle attacks. Mitigated by Dynamic ARP Inspection (DAI) on switches.

■ ***Quick Recall: ARP: broadcast 'Who has IP X?' → unicast reply with MAC; cached with TTL; ARP spoofing is a L2 MITM attack.***

### Q22. What is a VLAN? Why is it used?

A VLAN (Virtual Local Area Network) is a logical grouping of network devices into separate broadcast domains on the same physical switch infrastructure. Without VLANs, every port on a switch is in the same broadcast domain — every broadcast reaches every device.

How it works: IEEE 802.1Q standard adds a 4-byte tag to Ethernet frames containing a 12-bit VLAN ID (supporting 4094 VLANs). Switch ports are configured as either access ports (carry traffic for one VLAN, untagged) or trunk ports (carry traffic for multiple VLANs, tagged with 802.1Q).

Benefits: (1) Security — HR and Finance can be isolated on the same switch; (2) Broadcast reduction — smaller broadcast domains improve performance; (3) Flexibility — devices in the same VLAN can be on different physical switches (using trunks); (4) Cost savings — logical segmentation without additional hardware.

Inter-VLAN routing requires a Layer 3 device (router or L3 switch with 'router-on-a-stick' or SVIs — Switched Virtual Interfaces).

■ ***Quick Recall: VLAN = logical broadcast domain separation on shared switch; uses 802.1Q tagging; inter-VLAN needs L3 routing.***

### Q23. Explain the Network layer and its functions.

The Network layer (Layer 3) provides logical addressing and routing of packets between different networks — enabling end-to-end communication across the Internet. While the Data Link layer handles hop-to-hop delivery within a network, the Network layer handles the complete source-to-destination journey.

Core functions: (1) Logical Addressing — IP addresses identify hosts globally and hierarchically. (2) Routing — determining the best path from source to destination (using routing protocols: RIP, OSPF, BGP). (3) Forwarding — moving packets to the next hop based on the routing table. (4) Fragmentation — splitting packets too large for a link's MTU (only in IPv4; IPv6 does this only at the source). (5) Error reporting — ICMP provides error messages (unreachable, TTL exceeded, redirect).

The Network layer does NOT guarantee delivery, ordering, or reliability — these are Transport layer concerns. The Internet's design philosophy is to keep the network core simple (dumb pipes) and push intelligence to the edge (end hosts run TCP for reliability).

■ ***Quick Recall: Network layer: logical IP addressing, routing (path selection), forwarding, fragmentation, ICMP error reporting.***

### Q24. What is the difference between IPv4 and IPv6?

IPv4 uses 32-bit addresses, providing approximately 4.3 billion unique addresses. The explosion of internet-connected devices (smartphones, IoT) has exhausted IPv4 address space. NAT has temporarily mitigated this but adds complexity and breaks end-to-end connectivity.

IPv6 uses 128-bit addresses, providing 3.4 × 10^38 addresses — effectively unlimited. IPv6 addresses are written in hex with colons (e.g., 2001:0db8:85a3::8a2e:0370:7334). A double colon (::) represents one or more groups of all zeros.

Technical improvements: IPv6 has a fixed 40-byte header (easier processing), no header checksum (reduces per-hop processing), no fragmentation by routers, no broadcast (replaced by multicast + anycast), mandatory SLAAC (stateless address autoconfiguration), and IPsec originally designed as mandatory.

Transition mechanisms: Dual Stack (device runs both IPv4 and IPv6 simultaneously), Tunneling (IPv6 packets encapsulated in IPv4), NAT64/DNS64 (translation between IPv4 and IPv6 networks).

■ ***Quick Recall: IPv4=32-bit, 4B addresses, variable header; IPv6=128-bit, 340 undecillion addresses, fixed 40B header, no broadcast.***

### Q25. What are the different classes of IP addresses?

Classful addressing divides IPv4 address space into fixed-size classes determined by the leading bits. Class A: starts with 0, first octet range 1–126, default mask /8, ~16 million hosts per network. Class B: starts with 10, range 128–191, default mask /16, 65,534 hosts. Class C: starts with 110, range 192–223, default mask /24, 254 hosts. Class D: starts with 1110, range 224–239, reserved for multicast (no subnet mask concept). Class E: starts with 1111, range 240–255, experimental and reserved.

Special addresses: 127.0.0.0/8 (loopback, 127.0.0.1 = localhost), 169.254.0.0/16 (APIPA — Automatic Private IP Addressing, when DHCP fails), 255.255.255.255 (limited broadcast). Private ranges (RFC 1918): 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

Classful addressing has been superseded by CIDR (Classless Inter-Domain Routing), which allows flexible prefix lengths. However, understanding classes is essential for interviews because subnetting and private ranges still follow class boundaries.

■ ***Quick Recall: A=/8(1-126), B=/16(128-191), C=/24(192-223), D=multicast(224-239); private: 10/8, 172.16/12, 192.168/16.***

### Q26. What is subnetting? How do you calculate subnet mask and usable hosts?

Subnetting divides a larger network into smaller, more manageable sub-networks. This improves security (isolate departments), reduces broadcast traffic (smaller domains), and enables efficient IP address allocation.

Process: (1) Determine how many subnets needed → borrow bits from host portion. 2^(borrowed bits) = number of subnets. (2) Determine new subnet mask: original network bits + borrowed bits. (3) Block size = 256 - last non-255 octet of subnet mask. Subnets increment by block size.

Formulas: Number of subnets = 2^s where s = stolen bits. Hosts per subnet = 2^h - 2 where h = remaining host bits (-2 for network address and broadcast address). CIDR notation: /n means n network bits.

Example: 192.168.1.0/24, need at least 6 subnets → borrow 3 bits (2^3=8 subnets) → /27 (255.255.255.224). Block size = 256-224 = 32. Subnets: .0/27, .32/27, .64/27, .96/27, .128/27, .160/27, .192/27, .224/27. Hosts per subnet = 2^5-2 = 30.

■ ***Quick Recall: Subnets=2^borrowed; hosts=2^host_bits−2; block_size=256−mask_octet; CIDR /n = n network bits.***

### Q27. What is CIDR? How is it different from classful addressing?

CIDR (Classless Inter-Domain Routing, RFC 1519, 1993) abandoned the rigid class-based structure of IPv4 addressing. Instead, any prefix length can be used (e.g., /17, /21, /29), not just /8, /16, or /24.

Route aggregation (supernetting): multiple networks with a common prefix can be advertised as a single route. Example: 192.168.0.0/24 through 192.168.3.0/24 (4 networks) can be summarised as 192.168.0.0/22 — one routing table entry instead of four.

Classful addressing wastes addresses: a Class B gave 65,534 hosts even if you only needed 500 (wasting 65,000). CIDR allows exactly /23 (510 hosts) for that case. CIDR dramatically reduced the size of BGP routing tables that would otherwise have exploded with the growth of the Internet.

ISPs use CIDR blocks: IANA allocates large blocks to RIRs (ARIN, RIPE, APNIC); RIRs assign blocks to ISPs; ISPs assign smaller blocks to customers.

> ■ ***Quick Recall: CIDR = flexible prefix length (not just /8/16/24); enables route aggregation; reduces routing table size.***

### Q28. What is NAT? Explain Static NAT, Dynamic NAT, and PAT.

NAT (Network Address Translation) modifies IP address (and sometimes port) information in packet headers as packets pass through a router. It was created to conserve IPv4 address space — multiple private-IP devices share a single (or few) public IPs.

Static NAT: one-to-one permanent mapping. A specific private IP always maps to the same public IP. Used for servers that need a consistent inbound address (web server, mail server).

Dynamic NAT: the router has a pool of public IPs and dynamically assigns one to a private IP when it initiates a connection. The mapping lasts for the session. Cannot support more simultaneous external connections than there are public IPs in the pool.

PAT/NAPT (Port Address Translation): the most common form (used in all home routers). Multiple private IPs share a SINGLE public IP. The router uses port numbers to distinguish connections. Private IP:port → public IP:unique port. The router maintains a NAT table. Disadvantage: breaks end-to-end connectivity, complicates P2P and VoIP (requires NAT traversal with STUN/TURN).

> ■ ***Quick Recall: Static NAT=1:1 mapping; Dynamic NAT=pool of public IPs; PAT/NAPT=many:1 with port differentiation.***

### Q29. What is ICMP? What is Ping and Traceroute?

ICMP (Internet Control Message Protocol, RFC 792) is a Network-layer protocol used by IP hosts and routers to communicate error conditions and operational information. ICMP messages are encapsulated in IP packets with protocol number 1.

Key ICMP message types: Echo Request (Type 8) and Echo Reply (Type 0) — used by ping. Destination Unreachable (Type 3) — with subtypes for network/host/port/protocol unreachable. Time Exceeded (Type 11) — TTL expired; used by traceroute. Redirect (Type 5) — router tells host to use a better gateway.

Ping: sends ICMP Echo Requests to the target and measures RTT. Tests basic IP connectivity and measures latency. Firewalls may block ICMP — a failed ping doesn't necessarily mean the host is unreachable.

Traceroute (tracert on Windows): sends a series of packets with TTL=1, 2, 3, etc. When a router decrements TTL to 0, it discards the packet and sends an ICMP Time Exceeded back. This reveals each router's IP address and RTT. Unix traceroute uses UDP packets; Windows tracert uses ICMP Echo Requests.

■ ***Quick Recall: ICMP = error/diagnostic protocol in IP; ping uses Type 8/0 (Echo); traceroute uses TTL expiry to discover hops.***

### Q30. Explain Distance Vector routing and Link State routing. Compare RIP and OSPF.

Distance Vector routing: each router maintains a routing table with (destination, distance, next hop). Routers periodically share their complete routing table with their direct neighbours. Using the Bellman-Ford algorithm, each router updates its table based on neighbours' information. Problem: convergence is slow; count-to-infinity — when a link fails, routers may increment hop count indefinitely. Solutions: split horizon, poison reverse, hold-down timers.

RIP (Routing Information Protocol): uses hop count as metric (max 15; 16=unreachable). Sends full table updates every 30 seconds. Simple but doesn't scale — 15-hop limit, slow convergence, no VLSM support in RIPv1 (RIPv2 adds VLSM). Used only in small networks.

Link State routing: each router floods link-state advertisements (LSAs) describing its directly connected links to ALL routers. After convergence, every router has an identical map of the network. Uses Dijkstra's algorithm to compute shortest path tree. Converges much faster than distance vector.

OSPF (Open Shortest Path First): IP protocol 89, no transport layer. Uses 'cost' as metric (inversely proportional to link bandwidth). Supports hierarchical design with areas (Area 0 = backbone). Fast convergence (triggered updates). Supports VLSM. Used in enterprise and ISP networks.

■ ***Quick Recall: Distance Vector (RIP)=share tables with neighbors, Bellman-Ford, slow convergence; Link State (OSPF)=flood LSAs, Dijkstra, fast.***

### Q31. What is BGP? Why is it called a path vector protocol?

BGP (Border Gateway Protocol, RFC 4271) is the routing protocol that makes the Internet work. It is the protocol used to exchange routing information between Autonomous Systems (ASes) — large independently managed networks like ISPs (Comcast, AT&T;), cloud providers (AWS, Google), or enterprises.

Why path vector? Unlike distance vector (which only shares distance/metric to destinations) or link state (which shares topology), BGP shares the complete AS path — the sequence of autonomous systems a route has traversed. Example: 'To reach 8.8.8.0/24, you go through AS64500, AS15169.' This explicit path enables loop prevention (reject routes containing your own AS number) and policy-based routing (prefer routes through certain ASes).

eBGP (external BGP) is between different ASes. iBGP (internal BGP) is between routers within the same AS. BGP uses TCP port 179 for reliable session establishment. BGP route selection uses a complex set of attributes: Local Preference (highest preferred, for outbound traffic), AS Path length (shorter preferred), MED (Multi-Exit Discriminator, hints for inbound traffic), and others.

BGP is intentionally slow to converge — stability is prioritised over speed. A major BGP misconfiguration can affect routing for the entire Internet (BGP hijacking).

■ ***Quick Recall: BGP = inter-AS path vector protocol; shares AS paths (not metrics); TCP 179; enables policy-based Internet routing.***

### Q32. What is the Transport layer? What are its responsibilities?

The Transport layer (Layer 4) provides logical process-to-process communication using port numbers. While IP provides host-to-host delivery, the Transport layer extends this to application-to-application delivery — multiple applications on the same host can communicate simultaneously because each has a unique port number.

Core responsibilities: (1) Segmentation and Reassembly — large application messages are divided into segments (TCP) or datagrams (UDP) and reassembled at the destination. (2) Multiplexing/Demultiplexing — source port + dest port in the header allow the OS to route incoming data to the correct application. (3) Flow Control (TCP) — prevents sender from overwhelming receiver. (4) Error Control (TCP) — acknowledgements, sequence numbers, and retransmission ensure reliable delivery. (5) Congestion Control (TCP) — prevents a sender from overwhelming the network.

The Transport layer is the lowest layer that provides end-to-end communication. Network layer devices (routers) don't typically look at Transport layer headers (except for NAT and firewalls with stateful inspection).

■ ***Quick Recall: Transport layer: process-to-process via ports; segmentation; multiplexing; TCP adds reliability/flow/congestion control.***

### Q33. What is the difference between TCP and UDP?

TCP (Transmission Control Protocol) is connection-oriented, reliable, and ordered. Before data transfer, a 3-way handshake establishes a connection. TCP guarantees that all sent data arrives, in order, and without errors. If a segment is lost, TCP retransmits it. TCP implements flow control (receive window) and congestion control (cwnd). TCP header is 20+ bytes. TCP is used where data integrity is critical: HTTP/HTTPS, FTP, SSH, email.

UDP (User Datagram Protocol) is connectionless and unreliable. It simply sends datagrams without acknowledgement, ordering, or error recovery. The header is only 8 bytes. UDP is much faster than TCP for the same link capacity because there is no handshake, no ACKs, no retransmissions, and no congestion avoidance slowing down the sender.

Use case decision: if you can tolerate loss (streaming video — a dropped frame is fine) or if retransmission would arrive too late to be useful (VoIP — a retransmitted voice packet from 500ms ago is useless), use UDP. If you need reliable ordered delivery (file downloads, web pages, database queries), use TCP.

QUIC (used in HTTP/3) implements reliability and multiplexing on top of UDP — getting the best of both worlds while avoiding TCP's head-of-line blocking.

■ ***Quick Recall: TCP=reliable/ordered/flow+congestion control/20B header; UDP=connectionless/fast/no guarantee/8B header.***

### Q34. Explain the TCP header format.

The TCP header is at minimum 20 bytes (Data Offset = 5, meaning 5 × 32-bit words).

Source Port (16 bits): identifies the sending application's port. Destination Port (16 bits): identifies the receiving application. Sequence Number (32 bits): byte offset of the first byte of this segment in the data stream. Acknowledgement Number (32 bits): next byte number the receiver expects — implicitly ACKs all bytes up to this number. Data Offset (4 bits): header length in 32-bit words (minimum 5 = 20 bytes). Reserved (3 bits). Flags (9 bits): NS, CWR, ECE (congestion), URG (urgent data), ACK (ack number valid), PSH (deliver to application immediately), RST (abort connection), SYN (synchronise sequence numbers), FIN (no more data from sender). Window Size (16 bits): number of bytes the receiver is willing to accept (receive window — flow control). Checksum (16 bits): covers header + data. Urgent Pointer (16 bits): valid only if URG flag set. Options: variable length; includes MSS (Maximum Segment Size), Window Scale (wsopt), SACK (Selective Acknowledgement), Timestamps.

Key flags to know: SYN sets up connection, ACK acknowledges data, FIN gracefully closes, RST abruptly aborts (e.g., server receives a packet for a closed port).

■ ***Quick Recall: TCP header: src/dst ports, seq#, ack#, data offset, flags(SYN/ACK/FIN/RST/PSH), window size, checksum, options.***

### Q35. Explain the 3-way handshake in TCP. Why is it needed?

The TCP 3-way handshake establishes a connection by synchronising the initial sequence numbers (ISNs) of both sides and allocating resources.

Step 1 — SYN: Client sends a SYN segment with its ISN (say x). The SYN flag is set. Client enters SYN-SENT state.

Step 2 — SYN-ACK: Server receives SYN, enters SYN-RECEIVED state, responds with SYN+ACK. SYN flag set (server's ISN = y), ACK flag set (ack = x+1, acknowledging client's SYN). Server allocates connection resources.

Step 3 — ACK: Client receives SYN-ACK, enters ESTABLISHED state, sends ACK (ack = y+1, acknowledging server's SYN). Server moves to ESTABLISHED state upon receiving this ACK.

Why three-way? Two-way would not work: the server needs to confirm that the client can receive the server's SYN. Without the third ACK, the server doesn't know if its own SYN was received. Why not four-way? Three steps are sufficient for mutual ISN exchange. The third ACK from the client is the minimum needed to confirm both sides are ready. Three-way prevents old duplicate SYNs (from a previous aborted connection) from being mistaken for new connections.

> ■ ***Quick Recall: 3-way handshake: SYN(ISN=x) → SYN+ACK(ISN=y, ack=x+1) → ACK(ack=y+1); synchronises sequence numbers.***

### Q36. Explain the 4-way handshake in TCP connection termination.

TCP connection termination requires four steps because TCP is full-duplex. Each side must independently close its half of the connection. Either side (client or server) can initiate the close.

Step 1 — FIN: The active closer (say client) sends FIN to indicate it has no more data to send. Client enters FIN-WAIT-1. Note: the client can still receive data.

Step 2 — ACK: Server ACKs the FIN (ack = client_seq + 1). Client enters FIN-WAIT-2. Server enters CLOSE-WAIT — it can still send data to the client.

Step 3 — FIN: When the server has sent all its data, it sends its own FIN. Server enters LAST-ACK.

Step 4 — ACK: Client sends ACK (ack = server_seq + 1). Client enters TIME-WAIT for 2×MSL (Maximum Segment Lifetime, typically 2 minutes). After TIME-WAIT expires, client moves to CLOSED. Server moves to CLOSED upon receiving the ACK.

TIME-WAIT purposes: (1) Ensures the final ACK reaches the server — if the ACK is lost, the server retransmits FIN and the client (still in TIME-WAIT) can respond. (2) Ensures any delayed packets from the old connection have died off before a new connection reuses the same port pair.

> ■ ***Quick Recall: 4-way termination: FIN→ACK→FIN→ACK; TIME-WAIT=2×MSL ensures last ACK delivered and old packets expire.***

### Q37. What is TCP flow control? How does the sliding window work?

Flow control prevents the sender from transmitting faster than the receiver can process. The receiver advertises its available buffer space — the receive window (rwnd) — in the TCP header's Window Size field with every ACK.

Sliding window: the sender can have at most min(cwnd, rwnd) bytes of unacknowledged data outstanding. As ACKs arrive, the window slides forward. The sender can immediately send new data equal to the bytes just acknowledged (since the ACK 'frees up' space in the window).

Zero Window Probing: if rwnd drops to 0 (receiver buffer full), the sender stops. To detect when the receiver's buffer has space again (since the receiver's window-update segment could be lost), the sender periodically sends 1-byte 'probe' segments.

Silly Window Syndrome: if the receiver always sends tiny window advertisements (e.g., 1 byte), the sender sends tiny segments — wasteful overhead. Fixed on receiver side by Clark's solution (don't advertise small windows; only advertise when buffer is half-full or has space for a full MSS) and on sender side by Nagle's algorithm (buffer small data until a full MSS-worth is available or all outstanding data is acknowledged).

■ ***Quick Recall: Flow control: receiver advertises rwnd; sender can't exceed min(cwnd,rwnd) unACKed; zero window probing prevents deadlock.***

### Q38. What is TCP congestion control? Explain Slow Start and Congestion Avoidance.

TCP congestion control prevents the sender from overwhelming the network — not just the receiver. The sender maintains cwnd (congestion window) alongside rwnd. Effective window = min(cwnd, rwnd).

Slow Start: cwnd begins at 1 MSS (Maximum Segment Size, typically 1460 bytes). After each ACK, cwnd += 1 MSS. This results in cwnd doubling every RTT (exponential growth). Despite the name 'slow start,' cwnd grows exponentially — it's 'slow' only compared to immediately sending at full speed. Slow start continues until cwnd reaches ssthresh (Slow Start Threshold) or a loss event occurs.

Congestion Avoidance: when cwnd ≥ ssthresh, cwnd grows linearly — += MSS²/cwnd per ACK, which equals +1 MSS per RTT. This probes for additional bandwidth more cautiously.

Loss events: Timeout (severe) — ssthresh = cwnd/2, cwnd = 1 MSS, restart slow start. 3 duplicate ACKs (mild — packet lost but later packets received) — ssthresh = cwnd/2, cwnd = ssthresh (Fast Retransmit + Fast Recovery in Reno).

■ ***Quick Recall: Slow Start: cwnd doubles/RTT until ssthresh; Congestion Avoidance: linear +1MSS/RTT; loss halves ssthresh.***

### Q39. What is Fast Retransmit and Fast Recovery?

Fast Retransmit and Fast Recovery are optimisations that allow TCP to recover from isolated packet loss without waiting for the retransmission timeout (which can be hundreds of milliseconds or seconds).

Fast Retransmit: when a sender receives 3 duplicate ACKs for the same sequence number, it infers that the segment after the ACKed one was lost (subsequent segments arrived and triggered ACKs for the last in-order byte). Without waiting for the timeout timer, the sender immediately retransmits the likely-lost segment. The rationale: 3 dup ACKs indicate that later segments are arriving, so the network isn't congested — just one packet dropped.

Fast Recovery (TCP Reno): after fast retransmit, instead of dropping back to cwnd=1 (as Tahoe does), Reno sets ssthresh = cwnd/2 and cwnd = ssthresh + 3 MSS (the 3 accounts for the 3 segments that must have left the network, since they triggered dup ACKs). TCP enters Congestion Avoidance from this point — not Slow Start. This maintains higher throughput.

TCP Tahoe (older) treats 3 dup ACKs same as timeout — drops cwnd to 1. Reno's fast recovery is gentler and performs significantly better on links with occasional isolated packet loss.

■ ***Quick Recall: Fast Retransmit: 3 dup ACKs → retransmit immediately; Fast Recovery (Reno): ssthresh=cwnd/2, cwnd=ssthresh+3, skip slow start.***

### Q40. What is the TIME_WAIT state in TCP? Why is it needed?

TIME_WAIT is the state entered by the active closer (the side that sends the final ACK in the 4-way termination) after it sends that final ACK. The duration is 2×MSL (Maximum Segment Lifetime, defined as 2 minutes in RFC 793, giving 4 minutes of TIME_WAIT). The actual OS default is often 60 seconds in practice.

Why is it needed? Reason 1 — Reliability of the final ACK: The final ACK sent by the active closer might be lost. The passive closer will retransmit its FIN. If the active closer has already closed, it will respond with RST, which confuses the passive closer. By staying in TIME_WAIT, the active closer can respond correctly to a retransmitted FIN by resending the final ACK.

Reason 2 — Preventing phantom connections: Old packets from a previous connection with the same 4-tuple (src IP, src port, dst IP, dst port) might still be wandering the network. If a new connection immediately reuses the same 4-tuple, these old packets could be mistakenly accepted as part of the new connection. TIME_WAIT ensures all packets from the old connection have expired (lived their full MSL) before the 4-tuple can be reused.

Practical impact: Servers that close many short-lived connections can accumulate thousands of TIME_WAIT sockets. This is not a problem on most modern OSes (they handle it gracefully), but misconfigured servers with limited port ranges may run out of ephemeral ports.

> ■ ***Quick Recall: TIME_WAIT = 2×MSL after final ACK; ensures final ACK reaches server + old packets die before 4-tuple reuse.***

### Q41. What is a port number? What are well-known, registered, and dynamic ports?

A port number is a 16-bit unsigned integer (0–65535) that identifies a specific application or service on a host. Combined with an IP address, it forms a socket that uniquely identifies a process on the network. The Transport layer uses source and destination port numbers to multiplex/demultiplex data to the correct application.

Well-Known Ports (0–1023): assigned by IANA, require root/administrator privileges. Examples: HTTP=80, HTTPS=443, FTP=20/21, SSH=22, Telnet=23, DNS=53, DHCP=67/68, SMTP=25, POP3=110, IMAP=143, SNMP=161/162, RDP=3389.

Registered Ports (1024–49151): assigned by IANA for specific applications, but don't require root. Examples: MySQL=3306, PostgreSQL=5432, MongoDB=27017, Redis=6379, Elasticsearch=9200, RabbitMQ=5672, HTTP Alt=8080, HTTPS Alt=8443.

Dynamic/Ephemeral Ports (49152–65535): used by clients as source ports. The OS dynamically assigns one when a client initiates a connection. After the connection closes, the port is returned to the pool. Linux typically uses 32768–60999 by default (configurable in /proc/sys/net/ipv4/ip_local_port_range).

> ■ ***Quick Recall: Well-known: 0–1023 (HTTP=80, HTTPS=443, SSH=22); Registered: 1024–49151; Dynamic/Ephemeral: 49152–65535.***

### Q42. Explain the Application layer and its protocols.

The Application layer (Layer 7 in OSI, combined with Session/Presentation) provides the interface between user applications and the network. It defines the protocols that applications use to communicate.

Client-Server model: a server is always on, has a permanent IP, responds to requests. Clients initiate connections, may have dynamic IPs. Examples: HTTP web browsing, email (SMTP/IMAP), DNS, FTP.

Peer-to-Peer (P2P) model: arbitrary end hosts communicate directly — no always-on server. Examples: BitTorrent, Gnutella, Skype (originally). P2P is self-scaling: as more peers join, they add capacity.

Key Application layer protocols: HTTP/HTTPS (web), DNS (name resolution), SMTP/IMAP/POP3 (email), FTP (file transfer), SSH (secure shell), DHCP (IP assignment), SNMP (network management), NTP (time

synchronisation), Telnet (remote terminal, insecure, obsolete), SIP (VoIP signalling), WebSocket (real-time bidirectional).

The application layer does not concern itself with HOW data is transported — it relies on the Transport layer (TCP or UDP socket) for that.

■ ***Quick Recall: Application layer: provides network services to apps; client-server or P2P; HTTP, DNS, SMTP, FTP, SSH, DHCP.***

### Q43. How does DNS work? Explain recursive and iterative queries.

DNS translates domain names (www.example.com) to IP addresses. It is a distributed database organised in a hierarchy: Root servers → Top-Level Domain servers (.com, .org, .in) → Authoritative Name Servers for specific domains.

When you type www.example.com: (1) Browser checks its DNS cache. (2) OS checks its DNS cache and /etc/hosts. (3) OS sends a recursive query to the configured resolver (e.g., 8.8.8.8 or your ISP's DNS server). This is a recursive query — the resolver does all the work on the client's behalf.

The recursive resolver then uses iterative queries: it asks a root name server → gets a referral to the .com TLD server → asks .com TLD server → gets referral to example.com's authoritative NS → asks authoritative NS → gets the A record (93.184.216.34) → returns it to the client. Each of these steps is an iterative query (the resolver does each step itself and asks someone else if it doesn't know).

DNS caching: responses include a TTL (Time To Live). The resolver caches the answer for TTL seconds. All resolvers and clients cache responses. This dramatically reduces load on root and TLD servers. Negative caching: 'NXDOMAIN' (domain doesn't exist) is also cached for the SOA's minimum TTL.

■ ***Quick Recall: DNS: client→recursive resolver→(iterative) root→TLD→authoritative; responses cached with TTL; uses UDP 53.***

### Q44. What are the different HTTP methods? Which are idempotent?

HTTP methods define the intended action for a request. The key distinction between safe, idempotent, and neither:

Safe methods don't modify server state: GET (retrieve), HEAD (retrieve headers only), OPTIONS (discover capabilities), TRACE (diagnostic echo). Idempotent methods can be called multiple times with the same effect as once: GET, HEAD, PUT (replace), DELETE, OPTIONS (all safe methods are also idempotent). Non-idempotent: POST (each call creates a new resource — submitting a payment form twice charges twice), PATCH (not inherently idempotent — depends on whether it uses absolute or relative values).

REST API CRUD mapping: GET = Read, POST = Create, PUT = Replace (full update), PATCH = Update (partial), DELETE = Delete.

Interview gotcha: DELETE is idempotent — deleting an already-deleted resource returns 404 but the state is the same (the resource doesn't exist). PUT is idempotent — sending the same update twice results in the same state. POST is NOT idempotent — submitting a form twice creates two records.

■ ***Quick Recall: Safe: GET/HEAD/OPTIONS; Idempotent: GET/PUT/DELETE/HEAD/OPTIONS; Non-idempotent: POST; PATCH usually not.***

### Q45. Explain HTTP status codes: 200, 301, 302, 400, 401, 403, 404, 500, 502, 503.

HTTP status codes are 3-digit numbers indicating the result of a request. Categories: 1xx=Informational, 2xx=Success, 3xx=Redirect, 4xx=Client Error, 5xx=Server Error.

200 OK: standard success. Request fulfilled, body contains result. 301 Moved Permanently: resource permanently moved to URL in Location header. Client (and search engines) should update their references. Response is cached. 302 Found (Temporary Redirect): resource temporarily at different URL. Client should continue using original URL for future requests. Not cached by default. 400 Bad Request: malformed request syntax, invalid parameters, or too large. Client must fix the request before retrying. 401 Unauthorized (misleadingly named): actually means 'unauthenticated' — client must send credentials. 403 Forbidden: client is authenticated but lacks permission. Unlike 401, re-authenticating won't help. 404 Not Found: resource doesn't exist at this URL. May be permanent or temporary. 500 Internal Server Error: generic server-side failure; unhandled exception, null pointer, etc. 502 Bad Gateway: proxy/gateway received invalid response from upstream server. 503 Service Unavailable: server temporarily unable to handle requests (overloaded or maintenance); Retry-After header may indicate when to try again.

■ ***Quick Recall: 200=OK, 301=permanent redirect, 302=temporary, 400=bad request, 401=auth needed, 403=forbidden, 404=not found, 500=server error.***

### Q46. What is the difference between HTTP and HTTPS?

HTTP (HyperText Transfer Protocol) transmits data in plaintext over TCP port 80. Any intermediary (ISP, Wi-Fi router, proxy, malicious actor) on the network path can read or modify the content. This makes HTTP unsuitable for transmitting sensitive data.

HTTPS (HTTP Secure) = HTTP over TLS (Transport Layer Security). TLS wraps the HTTP connection in an encrypted tunnel. Port 443. HTTPS provides: (1) Confidentiality — data is encrypted; cannot be read by intermediaries. (2) Integrity — data cannot be modified in transit; TLS uses HMAC to detect tampering. (3) Authentication — the server's identity is verified via X.509 certificates signed by trusted Certificate Authorities (CAs). This prevents impersonation attacks.

How it works: after TCP connection, a TLS handshake occurs. The server presents its certificate. The client verifies the certificate chain up to a trusted root CA. A session key is established via asymmetric cryptography (RSA or Diffie-Hellman key exchange). All subsequent data is encrypted with this symmetric session key (AES-256-GCM).

HSTS (HTTP Strict Transport Security): response header that tells browsers to always use HTTPS for this domain for a specified period — prevents SSL stripping attacks.

■ ***Quick Recall: HTTPS = HTTP + TLS; provides confidentiality (encryption), integrity (HMAC), and server authentication (certificates).***

### Q47. Explain the TLS/SSL handshake process.

TLS 1.2 handshake (2 RTTs before data): (1) ClientHello: client sends TLS version, random number, list of supported cipher suites and compression methods. (2) ServerHello: server selects cipher suite, sends its random. (3) Certificate: server sends its X.509 certificate containing public key. (4) ServerHelloDone. (5) ClientKeyExchange: client verifies certificate, generates Pre-Master Secret, encrypts it with server's public key, sends it. Both sides derive Master Secret from Pre-Master Secret + both randoms. (6) ChangeCipherSpec + Finished: client signals switching to encrypted mode. (7) Server responds with ChangeCipherSpec + Finished. Data transfer begins.

TLS 1.3 handshake (1 RTT): client sends ClientHello WITH key_share in the first message (using ECDHE groups). Server responds with ServerHello, Certificate, Finished — all in one flight. No RSA key exchange. All cipher suites provide Forward Secrecy. Obsolete algorithms removed.

Certificate Validation: browser checks (1) certificate is signed by a trusted CA (chain of trust), (2) domain name matches, (3) certificate is not expired, (4) certificate is not revoked (CRL/OCSP). Certificate Transparency (CT) logs provide public audit trail of issued certificates.

■ ***Quick Recall: TLS 1.2: 2-RTT (ClientHello→ServerHello→Certificate→KeyExchange→Finished); TLS 1.3: 1-RTT with ECDHE key_share.***

### Q48. What is a cookie? How is it different from a session?

A cookie is a small piece of data (key-value pair) stored in the client's browser. It is set by the server via the Set-Cookie HTTP response header and sent back by the browser in the Cookie request header for subsequent requests to the same domain.

Cookies have attributes: Domain (which sites can access), Path (which URLs), Expires/Max-Age (persistence — session cookies expire when browser closes; persistent cookies have explicit expiry), Secure (only sent over HTTPS), HttpOnly (cannot be accessed by JavaScript — prevents XSS theft), SameSite (Strict/Lax/None — CSRF protection).

A server-side session stores user state on the server, identified by a session ID that is stored in a cookie. The actual data (user object, cart, permissions) lives on the server (in memory, Redis, or database). The cookie only holds the session ID.

Difference: Cookies are client-side storage (data in browser). Sessions are server-side storage (data on server, ID in cookie). JWT (JSON Web Token) is a modern alternative that encodes user claims directly in the token — stateless, can be validated without server-side lookup, but cannot be revoked without a token blacklist.

■ ***Quick Recall: Cookie = client-side key-value in browser; Session = server-side state; HttpOnly prevents JS access; SameSite prevents CSRF.***

### Q49. What is a firewall? What are its types?

A firewall is a security system that monitors and controls incoming and outgoing network traffic based on predefined security rules. It acts as a barrier between trusted internal networks and untrusted external networks.

Packet-Filtering Firewall (Stateless): examines individual packets based on IP addresses, ports, and protocol. Very fast. No awareness of connection state. Cannot detect attacks spread across multiple packets.

Stateful Inspection Firewall: tracks the state of active connections in a state table. Can distinguish unsolicited inbound packets (block) from packets that are part of an established outgoing session (allow). Standard in most enterprise deployments.

Application (Proxy) Firewall: acts as an intermediary — terminates connections and re-initiates them. Can inspect application-layer content (HTTP payloads, FTP commands). Provides deep inspection but high latency.

Next-Generation Firewall (NGFW): combines stateful inspection with deep packet inspection (DPI), application awareness (block specific apps like BitTorrent), user identity integration, SSL/TLS decryption, and integrated IPS. Examples: Palo Alto, Fortinet, Cisco Firepower.

■ ***Quick Recall: Firewall types: packet filter (stateless), stateful inspection, application proxy (L7), NGFW (DPI + IPS + identity).***

### Q50. What is a CDN? How does it improve website performance?

A CDN (Content Delivery Network) is a geographically distributed network of servers (Points of Presence / PoPs) that stores cached copies of web content close to end users.

How it works: (1) DNS resolution directs users to the nearest CDN PoP (using Anycast routing or GeoDNS). (2) The PoP serves cached static content (images, CSS, JS, videos) directly without hitting the origin server. (3) On cache miss, the PoP fetches from origin, caches it, and serves the user.

Performance benefits: (1) Reduced latency — content served from a server 20km away instead of 10,000km. (2) Reduced origin load — most requests served from edge cache; origin only handles cache misses and dynamic content. (3) Throughput — CDN servers are optimised and have high bandwidth. (4) Availability — if origin goes down, cached content continues to be served.

Additional features: TLS termination at the edge (reduces TLS RTT), DDoS absorption (massive distributed capacity absorbs attacks), compression (Brotli/gzip), HTTP/2+HTTP/3 push, image optimisation. Examples: Cloudflare, AWS CloudFront, Akamai, Fastly, Google Cloud CDN.

■ ***Quick Recall: CDN = distributed PoPs cache content near users; reduces latency, origin load, and improves availability + DDoS resilience.***

# CHAPTER 12: 25 MUST-PRACTICE PROBLEMS

The following table lists 25 curated practice problems covering key Computer Networks concepts. Work through all subnetting problems manually before your interview. For DSA problems involving network concepts, focus on graph algorithms (BFS/DFS for network traversal, shortest path for routing).

| # | Source | Problem / Topic | CN Topic | Key Concept | Reference URL |
|---|---|---|---|---|---|
| 1 | GFG | IP Addressing & Subnetting Practice Set 1 | IPv4 Subnetting | Network/broadcast address, usable hosts | geeksforgeeks.org/ip-addressing-subnettin |
| 2 | GFG | IP Addressing & Subnetting Practice Set 2 | VLSM | Variable Length Subnet Masking allocation | geeksforgeeks.org/ip-addressing-variable- |
| 3 | GFG | GATE CS 2014 — Subnetting Q | CIDR Subnetting | Identify subnet, hosts, broadcast given CIDR | geeksforgeeks.org/gate-gate-cs-2014-set- |
| 4 | GFG | OSI Model MCQ Practice | OSI Layers | Layer identification, PDU names, protocols | geeksforgeeks.org/osi-model-mcq |
| 5 | GFG | TCP/IP Model Questions | TCP/IP | Compare OSI vs TCP/IP layer mapping | geeksforgeeks.org/tcp-ip-model |
| 6 | GFG | Stop and Wait Protocol Efficiency | Flow Control | Calculate efficiency with given T_t and T_p | geeksforgeeks.org/stop-and-wait-arq |
| 7 | GFG | Go-Back-N Window Size Calculation | Sliding Window | Window size, seq numbers, throughput | geeksforgeeks.org/go-back-n-gbn-arq |
| 8 | GFG | Selective Repeat ARQ Problems | Sliding Window | SR window constraint, buffer analysis | geeksforgeeks.org/selective-repeat-arq |
| 9 | GFG | CSMA/CD Efficiency Problems | MAC Protocols | Utilisation with collision probability | geeksforgeeks.org/carrier-sense-multiple-a |
| 10 | GFG | CRC Error Detection Practice | Data Link | Compute CRC using generator polynomial | geeksforgeeks.org/modulo-2-binary-divisio |
| 11 | GFG | Hamming Code Practice Problems | Error Correction | Encode data, detect and correct bit errors | geeksforgeeks.org/hamming-code-in-comp |
| 12 | GFG | Routing Table Longest Prefix Match | Network Layer | Find next hop using longest prefix match | geeksforgeeks.org/longest-prefix-matching |
| 13 | GFG | Dijkstra Shortest Path for OSPF | Link State Routing | Run SPF on a network graph | geeksforgeeks.org/dijkstras-shortest-path- |
| 14 | GFG | Bellman-Ford for Distance Vector | Routing | Update routing table using Bellman-Ford | geeksforgeeks.org/bellman-ford-algorithm- |
| 15 | GFG | TCP 3-Way Handshake Sequence Numbers | TCP | Trace SYN/SYN-ACK/ACK with given ISNs | geeksforgeeks.org/tcp-3-way-handshake-p |
| 16 | GFG | TCP Congestion Control cwnd Problems | Congestion Control | Trace cwnd through Slow Start / Congestion Avoidance | geeksforgeeks.org/tcp-congestion-control |
| 17 | GFG | DNS Resolution Step-by-Step | Application Layer | Trace recursive/iterative query steps | geeksforgeeks.org/domain-name-system-d |
| 18 | LeetCode | #200 Number of Islands | Graph / Networking | BFS/DFS — analogous to network discovery | leetcode.com/problems/number-of-islands |
| 19 | LeetCode | #1971 Path Exists in Graph | Routing / Graph | Check connectivity — relates to routing reachability | leetcode.com/problems/find-if-path-exists-i |
| 20 | LeetCode | #1334 Find City Smallest Neigh. | Routing | Shortest path problem — Floyd-Warshall | leetcode.com/problems/find-the-city-with-th |
| 21 | LeetCode | #743 Network Delay Time | Link State Routing | Dijkstra on network graph = OSPF simulation | leetcode.com/problems/network-delay-time |
| 22 | LeetCode | #1514 Path with Max Probability | Routing | Modified Dijkstra for reliability routing | leetcode.com/problems/path-with-maximur |
| 23 | GFG | IPv6 Address Compression | IPv6 | Compress/expand IPv6 addresses correctly | geeksforgeeks.org/ipv6-address-compress |
| 24 | GFG | NAT & PAT Translation Table | Network Layer | Trace packets through PAT table | geeksforgeeks.org/network-address-transl |
| 25 | GFG | ARP Cache Poison Detection | Data Link Security | Identify ARP spoofing attack in trace | geeksforgeeks.org/arp-spoofing-or-arp-poi |

## Study Strategy for These Problems

- Work subnetting problems (#1–4) with pencil and paper; don't use a calculator until you are comfortable doing binary conversions mentally.
- For TCP problems (#15–16), draw timelines with arrows — visualising SYN/ACK exchanges and cwnd growth graphs will help you explain it in an interview.
- LeetCode graph problems (#18–22) are directly analogous to routing concepts. When you see 'shortest path', think Dijkstra (OSPF). When you see 'connectivity', think routing protocols.

• For GFG GATE problems, attempt under time constraints (90 seconds per MCQ) to simulate exam conditions.

• After solving each problem, explain your solution out loud as if you were in an interview — articulating the solution is as important as solving it.

---

## END OF COMPUTER NETWORKS REVISION GUIDE

*Good luck with your interview! Remember: depth over breadth — interviewers prefer candidates who truly understand TCP congestion control or subnetting over those who have memorised 200 facts superficially.*